

The Claude Certified Architect

— Foundations —

A Complete Study Guide

*Build production-grade AI systems with Claude Code, the
Claude Agent SDK, the Claude API, and the Model Context Protocol.*

Five exam domains • Six scenarios • Worked questions
Hands-on labs • Quick reference

Independent Study Guide

Edition 1.0 • 2026

The Claude Certified Architect — Foundations

A Complete Study Guide

Edition 1.0 • 2026

This book is an independent study guide created to help candidates prepare for the *Claude Certified Architect — Foundations* certification. It is not an official Anthropic publication and is neither endorsed by nor affiliated with Anthropic. Product names such as “Claude,” “Claude Code,” and “Model Context Protocol” are used only for identification and educational purposes and remain the property of their respective owners.

The certification outline referenced here was first published in draft form in early 2025; the certification launched publicly in March 2026. Product details reflect the platform as of 2026 and evolve quickly. Always confirm current specifics — exam format, scoring, pricing, and product behaviour — against official Anthropic documentation and the official registration page before relying on them.

The content is provided “as is,” without warranty of any kind, express or implied. The author and contributors accept no liability for any loss or outcome arising from its use.

All explanatory text is original. Sample-question wording is paraphrased; no copyrighted exam material is reproduced.

Prepared with care for learners.

Contents

About This Book	ii
Part 0 — The Exam at a Glance	1
Part 1 — Foundations: The Four Pillars	5
Part 2 — Domain 1: Agentic Architecture & Orchestration (27%)	14
Part 3 — Domain 2: Tool Design & MCP Integration (18%)	20
Part 4 — Domain 3: Claude Code Configuration & Workflows (20%)	25
Part 5 — Domain 4: Prompt Engineering & Structured Output (20%)	30
Part 6 — Domain 5: Context Management & Reliability (15%)	35
Part 7 — The Six Scenarios Decoded	39
Part 8 — Worked Sample Questions & Test-Taking Strategy	42
Part 9 — Hands-On Preparation Exercises	48
Part 10 — Quick Reference	57

About This Book

This is a comprehensive, self-contained study guide for the **Claude Certified Architect – Foundations** certification. It is organized as a book: read it front to back to learn the subject from the ground up, or jump to a specific chapter when you need to review one topic.

The certification is not a memorization test. It rewards **judgment** — knowing which of several reasonable-looking options is actually correct in a real production situation. Almost every exam question describes a realistic system that is misbehaving and asks, “What is the best fix?” The three wrong answers usually *sound* plausible. To choose well, you need to understand not just *what* each tool does but *why* and *when* you would reach for it. This guide is written to build exactly that kind of understanding.

Every chapter explains concepts in plain language first, then shows how they apply, then highlights the traps that the exam loves to test. Where the official exam outline is terse, this guide fills in the surrounding context using current Anthropic documentation and engineering guidance so that you understand the *whole* picture, not just the bullet point.

A note on currency. The technologies here evolve quickly. The certification outline was first published in early 2025; this guide reflects how the products work as of 2026. Where a name or detail has changed (for example, the *Task* tool is now also called the *Agent* tool, and custom slash *commands* have merged into *skills*), the guide teaches the concept the exam tests and then notes the current reality so you are never surprised.

How to Use This Guide

1. **Start with Part 0** to understand the shape of the exam — its format, scoring, domains, and the six scenarios that frame the questions.
2. **Read Part 1 (Foundations)** even if you have hands-on experience. It establishes the four pillars and the vocabulary used everywhere else.
3. **Work through Parts 2–6**, one per exam domain. Each chapter maps to a task statement in the official outline. Read the “Why it matters” and “Exam traps” boxes carefully — that is where points are won and lost.
4. **Study Part 7 (Scenarios)** to see how the abstract concepts combine into the concrete situations the exam describes.
5. **Do Part 8 (Worked questions)** to calibrate. Read each question, decide your answer *before* reading the explanation, then check your reasoning.
6. **Complete Part 9 (Hands-on exercises)** if you have time at a keyboard. Nothing cements this material like building it.
7. **Keep Part 10 (Quick reference)** open during your final review. It contains decision tables, a glossary, and cheat sheets.

Throughout, you will see recurring callouts:

- **In plain terms** — a simple restatement of a tricky idea.
- **Why it matters** — the production reason the concept exists.
- **Exam trap** — a common wrong answer and why it is wrong.
- **Remember** — a fact worth memorizing verbatim.

Part 0 — The Exam at a Glance

0.1 What the certification proves

The **Claude Certified Architect – Foundations** credential validates that you can make sound, tradeoff-aware decisions when designing and building real applications with Claude. It spans four core technologies:

- **Claude Code** — Anthropic’s agentic coding tool, used from the terminal, IDEs, the desktop app, and the web.
- **The Claude Agent SDK** — a library (Python and TypeScript) for building autonomous agents in your own programs.
- **The Claude API** — the underlying interface for sending messages, defining tools, and getting structured output.
- **The Model Context Protocol (MCP)** — the open standard for connecting Claude to your data and systems.

The emphasis is on **practical judgment**: architecture, configuration, and the tradeoffs that show up only in production. You are expected to understand both the *capabilities* and the *limits* of large language models in real systems.

0.2 Who the exam is for

The intended candidate is a **solution architect** who designs and ships production applications with Claude, typically with **6+ months** of hands-on experience. That person can comfortably:

- Build agentic applications with the Agent SDK (multi-agent orchestration, subagent delegation, tool integration, lifecycle hooks).
- Configure Claude Code for teams (CLAUDE.md files, Agent Skills, MCP servers, plan mode).
- Design MCP tools and resources to connect backend systems.
- Engineer prompts that produce reliable structured output (JSON schemas, few-shot examples, extraction patterns).
- Manage context across long documents, multi-turn conversations, and multi-agent handoffs.
- Integrate Claude into CI/CD for automated code review, test generation, and PR feedback.
- Make sound escalation and reliability decisions (error handling, human-in-the-loop, self-evaluation).

You do not need to be a researcher or know how the models are trained. The exam is about *building with* Claude, not building Claude.

0.3 Format, scoring, and how to pass

Aspect	Detail
Question type	Multiple choice. One correct answer, three distractors.

Aspect	Detail
What distractors are	Options a candidate with <i>incomplete</i> knowledge or experience might pick. They are designed to be tempting.
Scoring	Pass/fail against a standard set by subject-matter experts.
Scaled score	Reported on a 100–1,000 scale.
Passing score	720 .
Number of questions	60 scenario-based multiple-choice questions.
Time limit	120 minutes , proctored (online).
Cost	\$99 per attempt (free for the first 5,000 members of the Claude Partner Network at launch).
Guessing	No penalty for wrong answers, and unanswered questions count as incorrect — so always answer every question .
Scenarios	Each exam presents 4 scenarios , chosen at random from a pool of 6 . Questions are grouped under these scenarios.
Registration & prep	Registered through Anthropic’s Skilljar training platform; free Anthropic Academy preparation courses are open to everyone.

A note on currency. This guide’s *exam-logistics* details were originally taken from a **pre-launch draft** of the exam guide (Version 0.1, Feb 2025). The certification **publicly launched on March 12, 2026**, and the **question count (60)**, **time limit (120 minutes)**, and **cost (\$99/attempt)** above reflect that public launch. The **scaled-score (100–1,000) range and the 720 passing mark** come from the draft guide and may not match the final published values — confirm the current numbers on the official registration page before sitting the exam. The **five domains and their weights, the six scenarios, and all the conceptual and technical content in this guide remain accurate** against the launched exam’s published objectives.

In plain terms: Read every question to the end, eliminate the clearly wrong options, and pick the one that addresses the *root cause* with the *least added complexity*. Never leave a blank.

0.4 The five domains and their weights

The exam draws roughly these proportions of scored content from each domain. Budget your study time accordingly.

Domain	Topic	Weight
1	Agentic Architecture & Orchestration	27%
2	Tool Design & MCP Integration	18%
3	Claude Code Configuration & Workflows	20%
4	Prompt Engineering & Structured Output	20%
5	Context Management & Reliability	15%

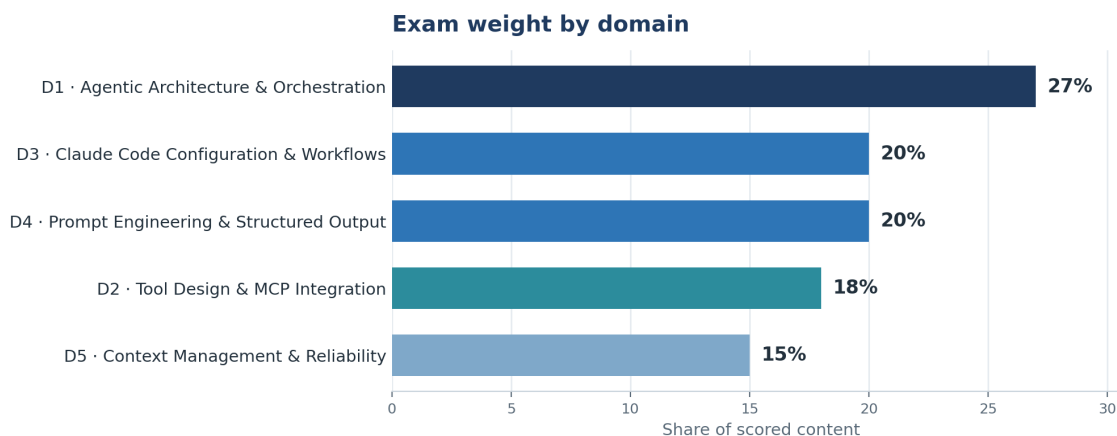


Figure 0.1 — Share of scored content by domain. Domain 1 alone is more than a quarter of the exam.

Domain 1 is the single largest slice, and Domains 3 and 4 are close behind. Together, agentic architecture, Claude Code, and prompt/structured-output skills make up about two-thirds of the exam.

0.5 The six scenarios

Every question lives inside a realistic scenario. Knowing the six in advance lets you anticipate the kinds of problems each will pose. (Part 7 decodes each in depth.)

1. **Customer Support Resolution Agent** — an Agent SDK agent handling returns, billing disputes, and account issues via MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`). Target: 80%+ first-contact resolution, with smart escalation.
2. **Code Generation with Claude Code** — using Claude Code for generation, refactoring, debugging, and docs; custom slash commands; `CLAUDE.md`; and choosing plan mode vs. direct execution.
3. **Multi-Agent Research System** — a coordinator delegating to specialized subagents (web search, document analysis, synthesis, report generation) to produce comprehensive, cited reports.
4. **Developer Productivity with Claude** — an Agent SDK tool that explores unfamiliar codebases, explains legacy systems, generates boilerplate, and automates chores using built-in tools (`Read`, `Write`, `Bash`, `Grep`, `Glob`) and MCP.
5. **Claude Code for Continuous Integration** — Claude Code embedded in CI/CD for automated code reviews, test generation, and PR feedback; prompts must be actionable and minimize false positives.
6. **Structured Data Extraction** — extracting data from unstructured documents, validating with JSON schemas, maintaining high accuracy, handling edge cases, and feeding downstream systems.

0.6 In-scope and out-of-scope topics

In scope (study these): agentic loops and `stop_reason` control flow; coordinator–subagent orchestration; subagent context management and crash recovery; tool interface design; MCP tools/resources/configuration; structured error responses; escalation decision-making; `CLAUDE.md` hierarchy and rules; custom commands and skills; plan mode; iterative refinement; structured output via tool use; few-shot prompting; batch processing; context-window optimization; human-review workflows; and information provenance.

Out of scope (don't waste time here): fine-tuning or training models; API authentication, billing, or account management; deep language/framework implementation beyond what tools and schemas require;

deploying or hosting MCP servers (infrastructure, networking, containers); Claude’s internal architecture or training; Constitutional AI / RLHF / safety training; embeddings and vector databases; computer use (browser/desktop automation); vision/image analysis; streaming API internals; rate limits, quotas, and pricing math; OAuth and key-rotation protocols; specific cloud-provider configs; benchmarking; prompt-caching internals (beyond knowing it exists); and tokenization specifics.

In plain terms: If a question feels like it is asking about infrastructure, billing, model training, or low-level protocol details, you are probably overthinking it — the exam stays at the level of *designing and configuring applications*.

Part 1 — Foundations: The Four Pillars

Before diving into the domains, you need a mental model of the four technologies and the single most important conceptual distinction in the whole exam: **workflows versus agents**.

1.1 The big picture: workflows vs. agents

Anthropic draws a clean line between two kinds of “agentic systems”:

- **Workflows** are systems where LLMs and tools are orchestrated through **predefined code paths**. You, the engineer, decide the steps. The model fills in the blanks at each step but does not choose the route.
- **Agents** are systems where the model **dynamically directs its own process** and tool usage. You give it a goal and tools; it decides what to do next, in what order, and when it is done.

In plain terms: A workflow is a railway — fixed tracks, predictable stops. An agent is a car with a destination — it picks the roads itself.

Why does this distinction matter so much? Because the right architecture depends on **how predictable the task is**:

- If you *can* predict the steps (e.g., “summarize, then translate, then format”), a workflow is simpler, cheaper, more reliable, and easier to debug.
- If you *cannot* predict the steps (e.g., “research this open-ended topic” or “fix whatever is breaking the tests”), an agent’s flexibility is worth its higher cost and the risk of compounding errors.

Anthropic’s guidance (“Building Effective Agents”) names **five composable patterns**, ordered from simplest to most autonomous. You will recognize all of them in the exam scenarios:

1. **Prompt chaining** — break a task into a fixed sequence of steps; each step’s output feeds the next. Good for predictable, multi-stage tasks (outline → draft → polish).
2. **Routing** — classify the input, then send it to a specialized handler. Good for separating concerns so one prompt does not have to be good at everything.
3. **Parallelization** — run work concurrently, in two flavors: *sectioning* (split a task into independent subtasks) and *voting* (run the same task several times and aggregate). Good for speed or for cross-checking.
4. **Orchestrator-workers** — a central model **dynamically** decomposes a task, delegates pieces to worker models, and synthesizes the results. The key difference from parallelization is that the subtasks are *not* known in advance; the orchestrator decides them based on the input. This is the foundation of the multi-agent research scenario.
5. **Evaluator-optimizer** — one model generates a result; another evaluates it and gives feedback; the loop repeats until the result is good enough. Good when you have clear evaluation criteria and iterative refinement adds measurable value.

Three principles guide all of this: **maintain simplicity**, **prioritize transparency** (show the model’s planning steps), and **carefully craft the agent-computer interface** (good tool documentation and testing). And the meta-rule: *start simple, add agentic complexity only when simpler solutions fall short*.

Exam trap: When a question describes a *predictable* multi-step task (like a code review that always analyzes files then checks integration), the right answer is usually a **fixed/prompt-chaining** decomposition — not a fully autonomous agent. Conversely, for *open-ended* investigation, dynamic decomposition wins. Match the pattern to the predictability of the task.

1.2 The Claude API and the agentic loop

The Claude API's core endpoint is **Messages**. You send a list of messages (alternating user and assistant roles), optionally a system prompt, optional tool definitions, and parameters like `max_tokens`. Claude responds.

The single most important field for agents is **stop_reason**, which tells you *why* Claude stopped generating on a turn. The values you must know:

- **tool_use** — Claude wants to call one or more tools. Your code must execute those tools and send the results back so Claude can continue.
- **end_turn** — Claude finished its response naturally. The loop is done.
- **max_tokens** — Claude hit the output token limit before finishing.
- **refusal** — Claude declined the request.

The **agentic loop** (sometimes “agent loop” or “tool loop”) is the cycle built around `stop_reason`:

1. Send the prompt (plus system prompt, tools, history) to Claude.
2. Inspect `stop_reason` on the response.
 - If `"tool_use"`: execute the requested tool(s), append the tool RESULTS to the conversation history, and go back to step 1.
 - If `"end_turn"`: stop. Present the final response.
3. Repeat until `end_turn`.

Two facts the exam tests repeatedly:

- **Tool results are appended to the conversation history** so the model can reason about what to do next. The loop is “stateful” only because you keep feeding the growing message list back in.
- The decision of *which* tool to call next is **model-driven**. Claude reasons about the situation; you do not hard-code a decision tree (unless you specifically need deterministic ordering — see Domain 1.4).

Remember: Continue the loop while `stop_reason == "tool_use"`. Terminate when `stop_reason == "end_turn"`. This is the canonical, correct control flow.

Exam trap (anti-patterns): Do **not** decide when to stop by (a) parsing Claude’s natural-language text for words like “done,” (b) setting an arbitrary iteration cap as your *primary* stopping mechanism, or (c) checking whether the response contains assistant text. Caps are fine as a *safety* limit, but the real stop signal is `end_turn`.

Tool use and tool_choice

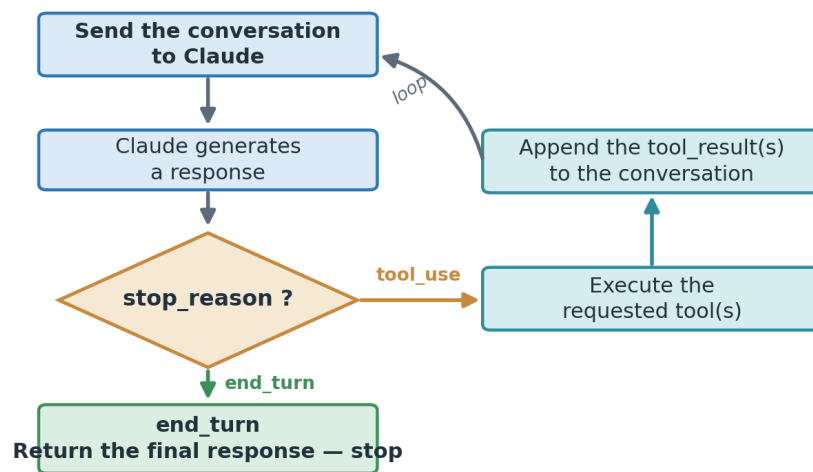
When you define tools, you can control whether and which tool Claude calls with **tool_choice**, which has four options:

tool_choice value	Behavior
"auto"	Claude decides whether to call a tool or reply with text. Default when tools are provided.
"any"	Claude must call <i>some</i> tool, but chooses which. Guarantees a tool call (no free-form text).
{"type": "tool", "name": "X"}	Claude is forced to call the specific tool X.
"none"	Claude may not call any tool. Default when no tools are provided.

(The exam outline emphasizes the first three; knowing all four is safest.)

Why it matters: tool_choice is the lever for *guaranteed structured output*. Forcing a specific extraction tool means Claude cannot wander off into prose — it must produce arguments matching that tool’s schema. More on this in Domain 4.

The agentic loop — control flow is driven by stop_reason



max_tokens or refusal also end the loop.

Figure 1.1 — The agentic loop. Each turn continues on tool_use and stops on end_turn.

1.3 The Claude Agent SDK

The **Claude Agent SDK** (formerly the *Claude Code SDK*) is a library, available in **Python and TypeScript**, that runs Claude Code’s autonomous agent loop *inside your own process*. The crucial contrast:

- With the **base API / Client SDK**, you implement the tool loop — you write the while stop_reason == "tool_use" code, execute tools, and resend results.
- With the **Agent SDK**, Claude handles the loop for you. You call something like query(prompt=...)

and iterate over a stream of messages while the SDK reads files, runs commands, calls tools, and repeats until done.

The Agent SDK gives you the same **built-in tools** that power Claude Code:

Category	Tools	Purpose
File operations	Read, Edit, Write	Read, modify, and create files
Search	Glob, Grep	Find files by pattern; search file contents with regex
Execution	Bash	Run shell commands, scripts, git
Web	WebSearch, WebFetch	Search the web; fetch and parse pages
Discovery	ToolSearch	Load tools on demand instead of preloading all schemas
Orchestration	Agent (a.k.a. Task), Skill, AskUserQuestion, task-tracking tools	Spawn subagents, invoke skills, ask the user, track tasks

Key building blocks you must know:

- **AgentDefinition** — the configuration for a subagent: its description, its prompt (system prompt), its allowed tools, and optionally its model and reasoning effort. This is how you define a specialized worker.
- **The Task/Agent tool** — the mechanism by which a coordinator spawns subagents. A coordinator's allowed tools **must include Task** (current SDKs surface this as `Agent` in tool-use blocks but still reference `Task` internally — check both names for compatibility).
- **Hooks** — callbacks that fire at specific points in the loop (before/after a tool runs, when the agent finishes, etc.). They run in *your* process and can intercept, modify, or block actions — giving you deterministic control. (Full treatment in Domain 1.5.)
- **Sessions** — every run has a session ID you can capture to **resume** later or **fork** into a parallel branch.
- **Permissions and limits** — `allowed_tools` auto-approve tools; `permission_mode` governs the rest; `max_turns` and `max_budget_usd` cap how long/expensive a run can get; `effort` controls reasoning depth.

In plain terms: The base API is a manual transmission (you shift gears in the loop). The Agent SDK is automatic (Claude shifts for you). Both go the same places; you choose how much you want to drive.

1.4 Claude Code

Claude Code is Anthropic's agentic coding tool. It runs in the terminal, in IDEs (VS Code, JetBrains), in a desktop app, on the web, and on mobile. Under the hood it uses the same agent loop as the Agent SDK; in fact, the Agent SDK is "Claude Code as a library."

For the exam, Claude Code shows up as a set of **configuration and workflow features** you must know how to use and *where to put*:

CLAUDE.md files — the project's always-on memory

A CLAUDE.md is a plain-Markdown file that Claude Code **loads automatically into context at the start of every session**, before you type anything. It is where you record the things you'd otherwise have to repeat in every conversation: coding conventions, the architecture in one paragraph, how to run the build and tests, libraries to prefer or avoid, and review expectations.

What makes it powerful is the **hierarchy** — multiple CLAUDE.md files layer together, from broad to specific:

- **User level** (~/.claude/CLAUDE.md) — your personal preferences, applied to *every* project on your machine. **Not shared** with anyone (it lives in your home directory, not the repo).
- **Project level** (.claude/CLAUDE.md, or a CLAUDE.md at the repo root) — team standards for this codebase. **Committed to version control and shared**, so a teammate who clones the repo inherits them with zero setup.
- **Directory level** (a CLAUDE.md inside a subfolder) — rules that apply only when working in that part of the tree (e.g., a frontend/CLAUDE.md for UI conventions).
- **(Current versions add)** an enterprise-managed policy layer and machine-local overrides.

When several apply, the **more specific scope wins / layers on top of the broader one** (roughly: managed → project → user). You can also keep files modular with the **@import** syntax — a small root CLAUDE.md that imports per-package standards only where relevant.

Why it matters (and a classic exam diagnostic): If a teammate reports that Claude is ignoring the team's standards, the usual cause is that the rules were placed at **user** scope (personal, unshared) instead of **project** scope. Remember the split: **project = shared/committed; user = personal/uncommitted**.

CLAUDE.md hierarchy — more specific / managed layers override broader ones

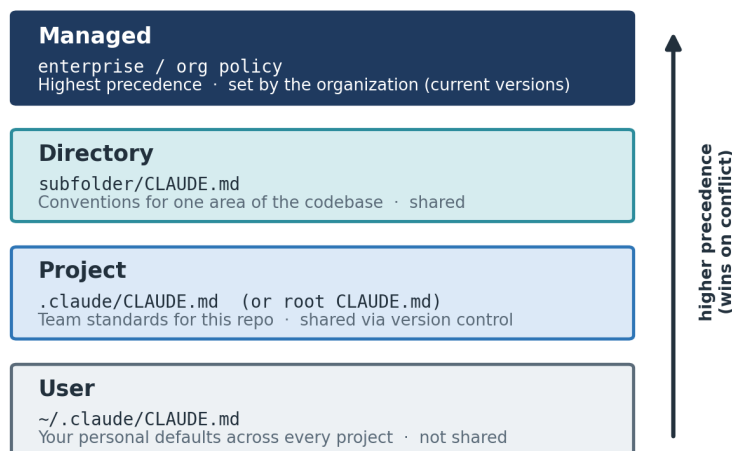


Figure 1.2 — The CLAUDE.md hierarchy. More specific and managed layers override broader ones.

.claude/rules/ — modular, path-scoped conventions

Instead of one ever-growing CLAUDE.md, you can break conventions into topic files inside a **.claude/rules/** directory (e.g., testing.md, api-error-handling.md). Each rule file can carry **YAML frontmatter** with a **paths** glob, so the rule loads **only when you're editing files that match**:

```
---
paths: ["**/*.test.*"]
---
```

Use Arrange–Act–Assert. Mock network calls. One behavior per test.

Because matching is by glob, a rule like ["**/*.test.*"] follows **test files wherever they live** in the tree. This is the right tool when conventions attach to a *kind* of file that's **scattered across directories** — something a single root `CLAUDE.md` (which relies on Claude *inferring* which section applies) and per-directory `CLAUDE.md` files (which are directory-bound and can't follow scattered files) both handle poorly. It also **saves context/tokens**, since a rule only enters the prompt when it's actually relevant.

Slash commands and skills — reusable workflows

These let you package a repeatable task once and trigger it as a `/command-name` shortcut (optionally taking arguments). - **Slash commands** historically lived as Markdown files in `.claude/commands/` (project-scoped, shared) or `~/.claude/commands/` (user-scoped, personal), and could accept parameters via an arguments placeholder. - These have now **converged with skills**, the recommended format: a folder at `.claude/skills/<name>/SKILL.md` that can bundle instructions plus helper scripts and resources. A `SKILL.md` uses frontmatter to control behavior: - **description** — *when* the skill should be used (this is how Claude decides to invoke it). - **context: fork** — run the skill in an **isolated sub-agent** so its verbose output never pollutes your main conversation; only the result comes back. - **allowed-tools** — restrict which tools the skill may use (e.g., read-only), preventing unintended destructive actions. - **argument-hint** — prompt the user for an argument if they invoke the skill without one.

The distinction the exam draws: Skills are on-demand and task-specific (loaded only when invoked); `CLAUDE.md` is always-loaded universal context. Reach for a skill to encapsulate a *procedure* you run sometimes; reach for `CLAUDE.md` for standards that should *always* apply.

Plan mode — explore and propose before touching code

In **plan mode**, Claude works **read-only**: it investigates the codebase, reasons about an approach, and presents a **step-by-step plan for your approval** *before* it edits a single file. You review (and correct) the plan up front, which prevents the expensive failure where the agent confidently makes dozens of wrong changes you then have to unwind.

Use it when complexity is real and **already evident**: large or multi-file changes, work with **several valid approaches**, or **architectural decisions** (e.g., where to draw a new service boundary). For a small, well-scoped change with an obvious fix — a single-file bug with a clear stack trace — **direct execution** is faster and plan mode is just ceremony. Plan mode often pairs with an **Explore subagent** that absorbs the noisy discovery work and returns a concise summary, protecting your main context budget.

Exam trap: When the requirements *already* describe architectural or multi-approach complexity, choose plan mode **now** — don't pick "start in direct execution and switch to planning later if it gets complicated."

MCP integration — connecting Claude Code to your systems

Claude Code can connect to **MCP servers** to gain tools and data from outside the codebase (issue trackers, databases, internal APIs). Configuration mirrors the shared-vs-personal split you've already seen: - `.mcp.json` — **project-scoped**, committed, shared with the team. Use `${VAR}` **environment-variable expansion** (e.g., `${GITHUB_TOKEN}`) so secrets are referenced, never hard-coded into the repo. - `~/.claude.json` — **user-scoped**, for your personal or experimental servers.

When a server connects, **all of its tools become available at once** (tools are *discovered at connection time*). Both shared and personal servers can be active in the same session. One practical note carried by the exam: write **rich MCP tool descriptions**, or Claude may default to a generic built-in (like Grep) instead of your more capable MCP tool.

Memory and context commands — keep long sessions healthy

Long sessions accumulate context, which both costs tokens and lets important early details drift. A few commands manage this: - **/memory** — inspect (and edit) which memory files are currently loaded, so you can *diagnose* why a convention is or isn't being followed. - **/compact** — compress the conversation so far into a summary, freeing space while keeping the key facts. (Claude Code also compacts **automatically** when context gets large, inserting a compaction boundary.) - **/clear** — reset the conversation when you're starting something genuinely unrelated.

Why it matters: Over a long run, models tend to drift toward “typical” patterns and lose the specifics discovered earlier. Compaction, scratchpad notes, and subagent isolation are the standard defenses (expanded in Domain 5).

CLI flags — driving Claude Code from scripts and CI

For automation (cron jobs, CI/CD pipelines, git hooks) you run Claude Code **non-interactively**: - **-p / -print** — the headless mode: take the prompt, run it, print the result to stdout, and **exit**. This is essential in CI, where an interactive prompt would otherwise **hang the job** waiting for input that never comes. - **--output-format json** — emit **machine-parseable** output so a pipeline can act on it programmatically (for example, posting review findings as inline comments on a pull request). - **--json-schema** — constrain that output to a schema you define. - **--append-system-prompt** — add instructions to the system prompt for that run (e.g., “only flag security issues”). - Session flags like **--resume / --continue** let an automated workflow pick up a prior named session.

Exam trap: A CI job that “hangs forever” is the tell-tale sign you forgot **-p / --print**. Invented flags such as `CLAUDE_HEADLESS` or `--batch` are common distractors.

Each of these features is explored at the task level — with the exact scenarios the exam builds on — in **Domain 3 (Part 4)**, and the MCP half in **Domain 2 (Part 3)**.

1.5 The Model Context Protocol (MCP)

MCP is an **open standard** (created at Anthropic, announced November 2024) for connecting AI applications to external data and tools. Instead of writing a bespoke integration for every system, you build against one protocol. Think of MCP as a universal adapter — “USB-C for AI tools.”

The architecture is a **client-server** model communicating over **JSON-RPC 2.0**:

- **Hosts** — the application the user interacts with (Claude Desktop, an IDE, Claude Code, your custom agent).
- **Clients** — live inside the host and each maintain a 1:1 connection to one MCP server.
- **Servers** — external programs that expose capabilities to the model.

MCP servers expose **three primitives**. Knowing who “controls” each is a frequent exam point:

Primitive	Controlled by	What it is	Example
Tools	Model	Executable functions the model can invoke to <i>take action</i>	<code>process_refund</code> , <code>query_database</code>
Resources	Application	Data sources that provide <i>context</i> (like read-only GET endpoints)	a database schema, a file's contents, an issue catalog
Prompts	User	Reusable templates for structuring interactions	a “format this document” template

Two more facts you must hold:

- **isError** — the MCP flag a tool uses to communicate that a call *failed* (rather than succeeded with data). How you populate the surrounding error payload is critical (Domain 2.2).
- **Tools are discovered at connection time** — when an MCP server connects, all its tools become available to the agent simultaneously. (This is why giving an agent too many servers/tools can hurt; see Domain 2.3.)

Remember the control mapping: Tools = model-controlled (actions). Resources = application-controlled (context/data). Prompts = user-controlled (templates). This single table answers several possible questions.

Model Context Protocol — hosts, clients, servers & the three primitives

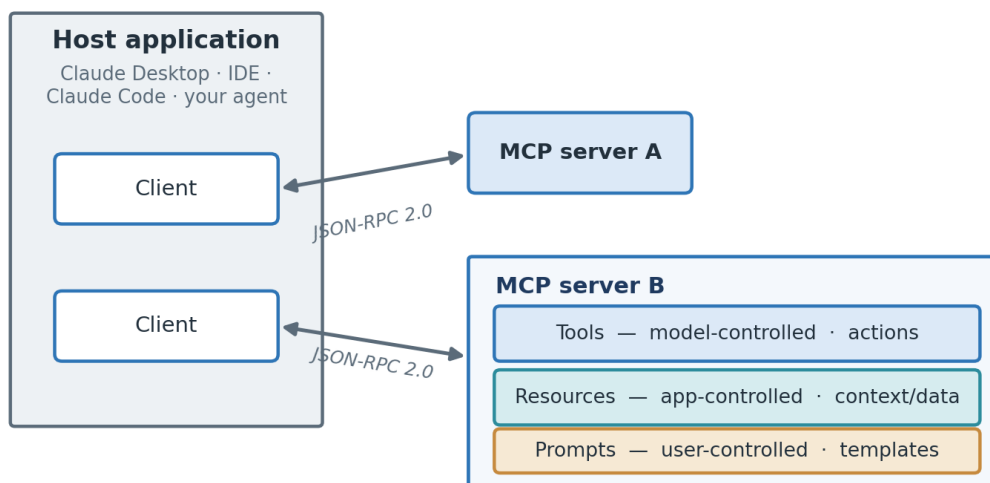


Figure 1.3 — MCP architecture: hosts contain clients; each client connects 1:1 to a server; servers expose tools, resources, and prompts.

1.6 How the pieces fit together

Here is the relationship in one breath:

- The **Claude API** is the engine: messages in, responses out, with tool use governed by `stop_reason` and `tool_choice`.
- The **Agent SDK** wraps that engine in a ready-made agent loop with built-in tools, subagents, hooks, sessions, and permissions — so you build agents without hand-writing the loop.
- **Claude Code** is the same engine and loop delivered as a developer tool, configured through `CLAUDE.md`, rules, skills, plan mode, and MCP.
- **MCP** is how all of the above reach *your* systems and data in a standard, reusable way.

The exam constantly asks you to choose the *right layer* for a problem: a prompt fix vs. a hook, a tool-description tweak vs. a new tool, plan mode vs. direct execution, the synchronous API vs. the batch API. The chapters ahead train that judgment.

Part 2 — Domain 1: Agentic Architecture & Orchestration (27%)

This is the largest domain. It is about *how agents run* (the loop), *how multiple agents cooperate* (orchestration), and *how to make critical steps reliable* (enforcement, hooks, sessions). The recurring exam theme: **prefer model-driven flexibility where appropriate, but use programmatic/deterministic mechanisms when correctness is non-negotiable.**

2.1 Designing agentic loops for autonomous task execution (Task 1.1)

The lifecycle

An agentic loop is the cycle introduced in Part 1: send a request → inspect `stop_reason` → if `tool_use`, run the tools and append their results → repeat → stop on `end_turn`. Each pass that includes tool calls is one **turn**. A simple question may take one turn; a complex task can chain dozens.

The reason the loop works is that **tool results are written back into the conversation history**. Claude does not “remember” tool outputs by magic — your code appends each tool result as a message, and the whole growing list is sent on the next request. That is how new information enters Claude’s reasoning.

Model-driven vs. pre-configured decisions

There are two ways to decide what happens next:

- **Model-driven** — Claude looks at the current context and decides which tool to call. This is the default agentic style and is right for tasks where the path depends on what is discovered.
- **Pre-configured** — you hard-code a decision tree or a fixed tool sequence. This is appropriate only when the order is genuinely fixed and must be guaranteed (and even then you often enforce it with code, not the loop itself — see 1.4).

Skills you must demonstrate

- Implement control flow that **continues on `tool_use` and stops on `end_turn`**.
- **Append tool results** between iterations so the model can incorporate them.
- **Avoid the anti-patterns.**

Exam trap — the three anti-patterns, restated: 1. *Parsing natural-language signals* (“the model said ‘finished’, so I’ll stop”). Models produce text in many contexts; this is unreliable. 2. *Arbitrary iteration caps as the primary stopping mechanism*. A cap is a safety net, not the logic that decides completion. 3. *Checking for assistant text content as a completion indicator*. Presence of text does not mean the task is done — Claude often emits text alongside tool calls.

The correct, robust signal is always `stop_reason == "end_turn"`.

2.2 Orchestrate multi-agent systems with coordinator-subagent patterns (Task 1.2)

When one agent cannot hold an entire complex task in its head, you split the work across a **coordinator** (a.k.a. orchestrator) and **subagents** (a.k.a. workers). This is the orchestrator-workers pattern from Part 1, realized as a **hub-and-spoke** architecture.

What the coordinator does

The coordinator is the hub. It: - **Decomposes** the overall task into subtasks. - **Delegates** each subtask to the right subagent. - **Routes** all inter-subagent communication through itself (subagents talk to the coordinator, not to each other). - **Aggregates** results. - **Decides which subagents to invoke** based on the query's complexity — it should *not* blindly run the full pipeline every time.

How subagents behave

This is the most-tested fact in the domain: **subagents run with isolated context**. A subagent does **not** automatically inherit the coordinator's conversation history. Each subagent starts fresh (it does load its own system prompt and project context like CLAUDE.md), does its focused job, and returns *only its final result* to the coordinator. The coordinator's context grows by that compact result, not by the subagent's entire transcript — which is exactly why subagents are such a powerful context-management tool.

Why hub-and-spoke?

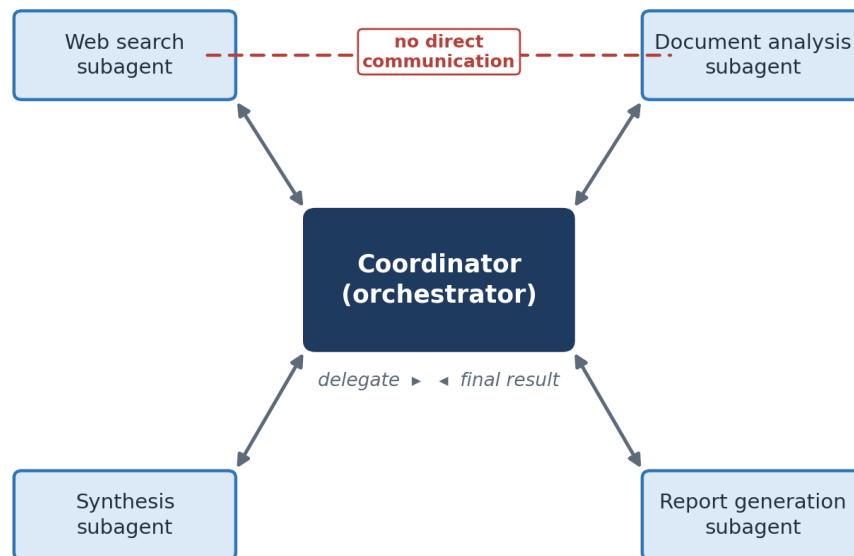
Routing everything through the coordinator gives you **observability** (one place to watch), **consistent error handling**, and **controlled information flow**. If subagents talked directly to each other, you would lose all three.

Risks and how to manage them

- **Overly narrow decomposition** — if the coordinator carves a broad topic into too-narrow pieces, the final output misses whole areas. (This is the exact failure in the famous “creative industries” sample question — the coordinator only generated visual-arts subtasks and missed music, writing, and film.)
- **Duplication** — if subagents overlap, you waste effort. **Partition the scope**: assign each subagent a distinct subtopic or source type.
- **Insufficient coverage** — build **iterative refinement loops**: the coordinator evaluates the synthesis for gaps, re-delegates targeted queries to search/analysis subagents, and re-runs synthesis until coverage is sufficient. (This is the evaluator-optimizer pattern applied to research.)

Exam trap: When a multi-agent system “succeeds” at every step but produces incomplete *output*, suspect the **coordinator's decomposition**, not the downstream agents. If each subagent did its assigned job correctly, the problem is *what they were assigned*.

Coordinator-subagent (hub-and-spoke) orchestration



All messages route through the hub. Each subagent has isolated context — only its final result returns.

Figure 2.1 — Coordinator-subagent (hub-and-spoke) orchestration. All traffic flows through the coordinator; each subagent has isolated context.

2.3 Configure subagent invocation, context passing, and spawning (Task 1.3)

Spawning subagents

- Subagents are spawned via the **Task tool** (current SDKs may surface it as **Agent**). For a coordinator to invoke subagents, its **allowedTools must include "Task"**. Omit Task from a subagent's own tools — **subagents cannot spawn their own subagents**.
- An **AgentDefinition** configures each subagent type: description (used to match tasks to the agent), prompt (its system prompt), and tools (its restricted toolset), plus optional model and effort.
- To run subagents **in parallel**, the coordinator emits **multiple Task tool calls in a single response** — not one per turn across separate turns. Parallel spawning is what turns a minutes-long sequential pipeline into a seconds-long concurrent one.
- **Fork-based session management** lets you branch from a shared analysis baseline to explore divergent approaches (e.g., compare two refactoring strategies). See 2.7.

Passing context (the part people get wrong)

Because subagents do **not** inherit parent context, you must **pass everything they need explicitly in their prompt**. Concretely:

- Put **complete findings from prior agents directly in the subagent’s prompt** — for example, hand the synthesis subagent the actual web-search results and document-analysis outputs it must combine.
- Use **structured formats that separate content from metadata** (source URLs, document names, page numbers) so attribution is preserved when context moves between agents.

Designing coordinator prompts

Write coordinator prompts that specify **research goals and quality criteria**, not rigid step-by-step procedures. Goal-oriented prompts let subagents adapt; over-specified procedural prompts make them brittle.

Remember: Subagents are amnesiacs by design. They know only their system prompt, their project context, and whatever you put in their prompt for this invocation. “It will just remember what the coordinator found” is always wrong.

2.4 Implement multi-step workflows with enforcement and handoff patterns (Task 1.4)

Programmatic enforcement vs. prompt-based guidance

This is one of the exam’s central tensions:

- **Prompt-based guidance** (telling Claude in the system prompt “always verify identity first”) is *probabilistic*. It works most of the time, but it has a **non-zero failure rate**.
- **Programmatic enforcement** (hooks, prerequisite gates) is *deterministic*. Code that blocks an action guarantees the rule, every time.

The decision rule: **when a business rule requires guaranteed compliance — especially anything with financial, legal, or safety consequences — use programmatic enforcement, not a prompt.**

Prerequisite gates

A prerequisite gate blocks a downstream tool until an upstream step has completed. The canonical example: **block process_refund until get_customer has returned a verified customer ID**. A prompt instruction (“verify the customer first”) will occasionally be skipped; a gate cannot be.

Decomposing multi-concern requests

A single customer message might contain several issues (“my order is late, I was double-charged, and I want to change my address”). The skill is to **decompose the request into distinct items, investigate each (often in parallel, sharing context), then synthesize one unified resolution** — rather than tackling only the first issue or producing three disjointed replies.

Structured handoff protocols

When the agent escalates to a human mid-process, that human usually **cannot see the conversation transcript**. So the agent must compile a **structured handoff summary**: customer ID, root-cause analysis, relevant amounts (e.g., refund amount), and a recommended action. A good handoff lets the human act immediately instead of re-investigating.

Exam trap: Faced with “the agent sometimes skips a required step,” the *best* answer is almost always a **programmatic prerequisite/gate**, not “improve the prompt” or “add few-shot examples.”

Prompts reduce the failure rate; they do not eliminate it. (See Worked Question 1.)

2.5 Apply Agent SDK hooks for tool call interception and data normalization (Task 1.5)

Hooks are callbacks that fire at defined points in the agent loop. They run in *your application process* (not in the model's context window), so they are deterministic and do not consume tokens. The hooks you should know:

Hook	Fires	Typical use
PreToolUse	Before a tool executes	Validate inputs; block dangerous or policy-violating calls
PostToolUse	After a tool returns	Transform/normalize outputs; audit; trigger side effects
UserPromptSubmit	When a prompt is sent	Inject extra context
Stop	When the agent finishes	Validate the result; save state
SubagentStart / SubagentStop	When a subagent spawns/completes	Track and aggregate parallel work
PreCompact	Before context compaction	Archive the full transcript first

Two flagship uses for the exam

1. PostToolUse for data normalization. Different MCP tools return data in different shapes — one returns Unix timestamps, another ISO 8601 strings, another numeric status codes. A PostToolUse hook can **normalize these heterogeneous formats into a consistent shape before the model processes them**, so the agent reasons over clean, uniform data.

2. PreToolUse (tool-call interception) for compliance. A hook can **intercept an outgoing tool call and block it if it violates a rule** — for example, refuse any process_refund over \$500 — and **redirect to an alternative workflow** such as human escalation.

The deciding principle

Use hooks **for deterministic guarantees**; use prompts only for probabilistic guidance. When the business rule must hold every single time, choose a hook.

Why it matters: A prompt that says “never refund more than \$500” will eventually let a \$501 refund through. A PreToolUse hook that checks the amount and blocks it will not. The exam rewards recognizing when you have crossed from “nice to have” into “must be guaranteed.”

2.6 Design task decomposition strategies for complex workflows (Task 1.6)

The core choice: **fixed sequential pipelines (prompt chaining)** vs. **dynamic adaptive decomposition**.

- **Prompt chaining** — break the task into a *predetermined* sequence of focused steps. Use it for **predictable, multi-aspect** work. Example: a code review that (1) analyzes each file individually, then

- (2) runs a separate cross-file integration pass.
- **Dynamic decomposition** — generate subtasks based on what is discovered at each step. Use it for **open-ended investigation**. Example: “add comprehensive tests to a legacy codebase” — first map the structure, identify high-impact areas, then build a prioritized plan that adapts as dependencies surface.

A key technique: splitting reviews to avoid attention dilution

When reviewing many files at once, a model’s attention spreads thin: some files get deep feedback, others superficial, bugs are missed, and feedback becomes contradictory (a pattern flagged in one file is approved in another). The fix is **per-file local analysis passes plus a separate cross-file integration pass**. Each file gets focused attention; integration issues get their own dedicated pass.

Exam trap: For inconsistent results on a large multi-file review, the right answer is to **split into focused passes** — *not* to “use a bigger context window,” “make developers split the PR,” or “run three passes and require 2-of-3 consensus.” A larger context window does not fix attention *quality*; consensus voting suppresses real bugs that are caught only intermittently. (See Worked Question 12.)

2.7 Manage session state, resumption, and forking (Task 1.7)

Agent work often spans multiple sittings or needs to branch. The SDK and Claude Code give you three tools:

- **Named session resumption** — `--resume <session-name>` continues a specific prior conversation, restoring its full context (files read, analysis done, actions taken).
- **fork_session** — creates an **independent branch from a shared baseline** so you can explore divergent approaches without disturbing the original (e.g., compare two testing strategies from the same codebase analysis).
- **Starting fresh with a structured summary** — sometimes the cleaner choice.

When to resume vs. start fresh

- **Resume** when the prior context is **mostly still valid**.
- **Start fresh with an injected summary** when the prior **tool results are stale** — for instance, after files have changed, the cached file contents in the old session are now wrong. A new session seeded with a structured summary of the relevant facts is **more reliable than resuming with stale results**.
- If you do resume after code changes, **tell the agent exactly which files changed** so it re-analyzes only those, rather than forcing a full re-exploration.

Remember: Stale tool results are a trap. If the world has moved on since the session was captured, prefer a fresh session with a clean, structured summary over resuming with outdated data. Forking is for *parallel exploration*, not for recovering from staleness.

Part 3 — Domain 2: Tool Design & MCP Integration (18%)

Tools are the **contract between a deterministic system (your code) and a non-deterministic agent (Claude)**. Unlike traditional APIs built for human developers, agent tools must assume the caller might misread, misuse, or hallucinate. This domain is about writing tools the agent can use *reliably*, reporting errors so the agent can *recover*, distributing tools so selection stays *accurate*, and wiring up MCP servers correctly.

3.1 Design effective tool interfaces with clear descriptions and boundaries (Task 2.1)

Descriptions are the steering wheel

Tool descriptions are the primary mechanism Claude uses to choose a tool. When descriptions are minimal, the model lacks the context to distinguish similar tools and selection becomes unreliable. A good description includes:

- The tool's **purpose** and when to use it (and when *not* to).
- **Input formats** it accepts, with **example queries**.
- **Edge cases** and **boundaries** — explicitly how it differs from similar tools.

Anthropic's own engineering work found that **even small refinements to tool descriptions yield dramatic improvements** in agent accuracy. Treat description-writing as real engineering, not documentation.

The overlap problem

Two tools with near-identical descriptions cause **misrouting**. Classic example: `analyze_content` vs. `analyze_document` both described as "analyzes things." Claude cannot tell them apart and picks wrong. Fixes:

- **Rename and re-describe to remove overlap** — e.g., rename `analyze_content` to `extract_web_results` with a web-specific description.
- **Split a generic tool into purpose-specific tools** with defined input/output contracts — e.g., split `analyze_document` into `extract_data_points`, `summarize_content`, and `verify_claim_against_source`.

Naming and parameters

Follow Anthropic's tool-design guidance: - **Namespace/prefix** related tools so their domain is clear. - Use **unambiguous parameter names**: prefer `user_id` over `user` (which could mean a name, an object, or an ID). - **Return meaningful, human-readable context** rather than opaque technical IDs. - **Prefer search-style tools** (`search_contacts`) over list-everything tools (`list_contacts`), and **manage token volume** with pagination, filtering, and truncation.

Watch the system prompt

Tool selection is **keyword-sensitive**. Wording in the *system prompt* can create unintended associations that override even well-written tool descriptions (e.g., a system prompt that keeps saying “document” nudges Claude toward the document tool inappropriately). When diagnosing misrouting, **review the system prompt for keyword bias**, not just the tool descriptions.

Exam trap: When two similar tools are confused and both have *minimal* descriptions, the best *first* step is to **expand the descriptions** (inputs, examples, edge cases, boundaries) — a low-effort, high-leverage fix. Few-shot examples add token overhead without fixing the root cause; a routing layer is over-engineering; consolidating tools is valid but heavier than a “first step.” (See Worked Question 2.)

3.2 Implement structured error responses for MCP tools (Task 2.2)

When a tool fails, *how* it tells the agent determines whether the agent can recover intelligently. MCP signals failure with the **isError** flag — but the flag alone is not enough.

Error categories

Distinguish the kinds of failure, because the right reaction differs:

Category	Meaning	Right reaction
Transient	Timeout, service temporarily unavailable	Retry (often locally)
Validation	Invalid input	Fix the input; do not blindly retry
Business	Policy violation (e.g., refund over limit)	Do not retry; explain to the user/escalate
Permission	Not authorized	Do not retry; needs different handling

Why uniform errors are harmful

A generic “Operation failed” tells the agent nothing. It cannot decide whether to retry, fix input, or give up — so it wastes retries on non-retryable failures or abandons recoverable ones. **Uniform error responses prevent appropriate recovery decisions.**

What a good error response contains

- An **errorCategory** (transient / validation / business / permission).
- An **isRetryable** boolean (or `retriable: false` for business-rule violations).
- A **human-readable description** — and for business errors, a **customer-friendly explanation** so the agent can communicate appropriately.

Local recovery and partial results

- Subagents should **recover locally** from transient failures and only **propagate to the coordinator the errors they cannot resolve**, including **what was attempted** and any **partial results**.

- **Distinguish access failures from valid empty results.** A timeout (access failure) needs a retry decision; “the query ran fine and found zero matches” (valid empty result) is a *success* with no data. Conflating them leads to bad recovery.

Remember: Structured, categorized, retryable-tagged errors let the agent make smart recovery decisions. Generic failures blind it. This idea reappears in Domain 5.3 (error propagation across agents).

3.3 Distribute tools appropriately across agents and configure tool choice (Task 2.3)

Fewer tools, better selection

Giving an agent too many tools degrades selection reliability. An agent with 18 tools makes worse choices than one with 4–5, because every extra tool increases decision complexity. And agents handed tools **outside their specialization** tend to misuse them (a synthesis agent that has web-search tools will start doing web searches it should delegate).

The principle is **scoped tool access**: give each agent only the tools its role needs, with a few **limited cross-role tools** for specific high-frequency needs.

Concrete moves

- **Restrict each subagent’s toolset** to its role to prevent cross-specialization misuse.
- **Replace generic tools with constrained alternatives** — e.g., replace `fetch_url` with `load_document` that validates document URLs.
- **Provide scoped cross-role tools** for common needs — e.g., give the synthesis agent a narrow `verify_fact` tool for simple lookups, while routing *complex* verification through the coordinator. (This is the principle of least privilege: satisfy the common case cheaply, keep the existing pattern for the hard case. See Worked Question 9.)

tool_choice for control flow

- Use **forced selection** (`tool_choice: {"type": "tool", "name": "..."}`) to guarantee a specific tool runs **first** — e.g., force `extract_metadata` before any enrichment tools — then handle subsequent steps in follow-up turns.
- Use **tool_choice: "any"** to guarantee the model **calls some tool** rather than returning conversational text.

Exam trap: When a specialized agent occasionally strays into another agent’s job, the fix is to **scope its tools** (and optionally add a *narrow* cross-role tool for the frequent simple case) — not to give it the full toolset of the other agent. Over-provisioning violates separation of concerns.

3.4 Integrate MCP servers into Claude Code and agent workflows (Task 2.4)

Scoping: project vs. user

- **Project-level servers** go in `.mcp.json` at the repo root. These are **shared with the team via version control** — the right place for tooling everyone needs.
- **User-level servers** go in `~/.claude.json`. These are **personal/experimental** and not shared.

Credentials without committing secrets

`.mcp.json` supports **environment variable expansion** — e.g., reference `${GITHUB_TOKEN}` so the actual token is read from the environment at runtime and never committed to source control.

Discovery and resources

- **All configured MCP servers’ tools are discovered at connection time** and become available simultaneously. (Hence the “too many tools” caution in 2.3.)
- **MCP resources expose content catalogs** — issue summaries, documentation hierarchies, database schemas — giving the agent *visibility into available data* so it can skip exploratory tool calls. Use **resources for context/data** and **tools for actions**.

Making MCP tools win over built-ins

Claude sometimes prefers a built-in tool (like Grep) over a more capable MCP tool. The fix is to **enhance the MCP tool’s description** to clearly explain its capabilities and outputs, so the agent understands it is the better choice.

Build vs. reuse

Prefer existing community MCP servers for **standard integrations** (e.g., Jira), and reserve **custom servers for team-specific workflows**. Don’t rebuild what already exists.

Remember the file map: `.mcp.json` (project, shared, version-controlled) vs. `~/.claude.json` (user, personal). Use `${VAR}` expansion for secrets. Resources = catalogs/context; tools = actions.

3.5 Select and apply built-in tools effectively (Task 2.5)

The Agent SDK and Claude Code ship file/search/execution tools. Knowing **which tool for which job** is testable:

Tool	Use it for
Grep	Searching file contents for patterns — function names, error messages, import statements
Glob	Finding files by name/extension pattern — e.g., <code>**/*.test.tsx</code>
Read	Loading full file contents
Write	Creating or overwriting files
Edit	Targeted modifications using unique text matching
Bash	Running shell commands, scripts, git

Key skills

- **Edit’s fallback.** Edit works by matching a *unique* anchor string. When the anchor text is **not unique**, Edit fails — the reliable fallback is **Read the full file, then Write the modified version**.
- **Explore incrementally.** Build codebase understanding by **starting with Grep to find entry points, then Read to follow imports and trace flows** — rather than reading every file upfront (which burns context).
- **Trace usage across wrappers.** To follow a function through wrapper modules, first **identify all exported names**, then **search for each name** across the codebase.

Exam trap: “Find all callers of a function” → **Grep** (content search). “Find all test files” → **Glob** (filename pattern). “Edit can’t find a unique match” → **Read + Write**. Don’t confuse content search (Grep) with filename search (Glob).

Part 4 — Domain 3: Claude Code Configuration & Workflows (20%)

This domain is about **configuring Claude Code for real teams**: where to put instructions so the right people get them, how to build reusable commands and skills, when to plan vs. just do it, how to iterate effectively, and how to run Claude Code in automated pipelines. Many questions hinge on **scope** — *who* sees a setting and *when* it loads.

4.1 Configure CLAUDE.md files with appropriate hierarchy, scoping, and modular organization (Task 3.1)

The hierarchy

CLAUDE.md files give Claude persistent instructions, loaded at the **start of every session**. They are layered:

- **User-level** — `~/ .claude/CLAUDE.md`. Applies to *all your projects*, on *your machine only*. **Not shared with teammates via version control**.
- **Project-level** — `.claude/CLAUDE.md` or a root `CLAUDE.md`. Committed to the repo and shared with everyone who clones it.
- **Directory-level** — a `CLAUDE.md` inside a subdirectory, scoped to that part of the tree.

In current Claude Code, the hierarchy is richer (an enterprise-managed/policy layer and a local `CLAUDE.local.md` also exist), and the precedence runs roughly **managed > local > project > user**. The rule to memorize: **more specific instructions win over broader ones**. If user-level says “2-space indent” and project-level says “4-space indent,” the **project** rule wins inside that project.

Modular organization

- **@import syntax** — keep `CLAUDE.md` modular by referencing external files: `@docs/coding-standards.md`. Imports are resolved relative to the file containing them, can nest (to a depth limit), and trigger an approval prompt the first time they pull from an external location. Use imports to **selectively include the standards relevant to each package** based on what its maintainers know matters.
- **.claude/rules/** — instead of one giant `CLAUDE.md`, split topics into focused files like `testing.md`, `api-conventions.md`, `deployment.md`. Files here are picked up automatically alongside `CLAUDE.md`.

Diagnosing problems

- A common failure: **a new teammate doesn’t get the instructions** because they were placed in **user-level** config (personal, not shared) instead of **project-level** (committed). The fix is to move them to project scope.
- Use the `/memory` command to **see which memory files are loaded** and to diagnose inconsistent behavior across sessions. (If an instruction “disappeared,” it may have been given only in conversation, or it lives in a nested `CLAUDE.md` that has not reloaded.)

Exam trap: “Why didn’t my teammate get this rule?” → it is in **user-level** config, which is not version-controlled. Put shared rules in **project-level** CLAUDE.md (or .claude/rules/). And remember: when levels conflict, **the more specific level wins**.

4.2 Create and configure custom slash commands and skills (Task 3.2)

Where commands and skills live

- **Project-scoped commands** — .claude/commands/ — shared via version control, available to everyone on the team.
- **User-scoped commands** — ~/.claude/commands/ — personal.
- **Skills** — .claude/skills/<name>/SKILL.md (project) or ~/.claude/skills/ (personal). A skill is a folder with a SKILL.md plus optional supporting files/scripts.

Current reality: Custom slash *commands* have **merged into skills**. Files in .claude/commands/ still work, but the recommended approach is .claude/skills/. Both create /command-name shortcuts. The exam’s framing of “commands in .claude/commands/” remains valid for answering scope questions.

SKILL.md frontmatter you must know

A SKILL.md begins with YAML frontmatter that controls behavior:

- **context: fork** — run the skill in an **isolated sub-agent context** so its (often verbose) output does **not pollute the main conversation**. Perfect for codebase-analysis or brainstorming skills.
- **allowed-tools** — **restrict** which tools the skill may use during execution (e.g., limit to file-write operations to prevent destructive actions).
- **argument-hint** — prompt the developer for required parameters when they invoke the skill without arguments.

(Current versions add more frontmatter — disable-model-invocation, user-invocable, agent, paths — but the three above are the exam’s focus.)

Skills vs. CLAUDE.md

- **Skills** are **on-demand**: invoked for a specific task (manually with /name, or auto-invoked by Claude when relevant).
- **CLAUDE.md** is **always-loaded**: universal standards that apply to every session.

Choose skills for *task-specific workflows* and CLAUDE.md for *universal standards*.

Exam trap: A team-wide command that should be available to everyone who clones the repo goes in **.claude/commands/** (project-scoped, version-controlled) — **not** in ~/.claude/commands/ (personal), **not** in CLAUDE.md (that is for context/instructions, not command definitions), and there is **no** .claude/config.json commands array. (See Worked Question 4.)

4.3 Apply path-specific rules for conditional convention loading (Task 3.3)

`.claude/rules/` files can carry a **YAML frontmatter `paths` field with glob patterns**, so a rule loads only when you edit a matching file. This:

- **Reduces irrelevant context and token usage** (rules don't load when they don't apply).
- **Spans directories**, which directory-level `CLAUDE.md` cannot. A glob like `**/*.test.tsx` applies to *all* test files no matter where they live.

Skills you must demonstrate: - Create rules with path scoping — e.g., `paths: ["terraform/**/*"]` so Terraform conventions load only when editing Terraform files. - Use glob patterns to apply conventions **by file type regardless of directory** — e.g., `**/*.test.tsx` for all React test files. - **Choose path-specific rules over subdirectory `CLAUDE.md`** when conventions must apply to files **scattered across the code-base**.

Exam trap: When test files (or any file type) are **spread throughout** the codebase and need consistent, *automatic* conventions, the answer is **`.claude/rules/` with glob paths** — not a per-directory `CLAUDE.md` (which is directory-bound), not “rely on Claude to infer from headers in one big `CLAUDE.md`,” and not skills (which require invocation rather than automatic, path-based loading). (See Worked Question 6.)

4.4 Determine when to use plan mode vs. direct execution (Task 3.4)

Plan mode

Plan mode lets Claude **explore read-only and design an approach before changing anything**. It is built for: - **Large-scale changes** and **multi-file modifications**. - **Multiple valid approaches** and **architectural decisions**. - Situations where **committing to the wrong approach would be costly to undo**.

Examples: microservice restructuring, a library migration touching 45+ files, or choosing between integration approaches with different infrastructure requirements.

Direct execution

Direct execution is right for **simple, well-scoped changes** with a clear path: a single-file bug fix with a clear stack trace, adding one validation conditional. Planning here would be overhead.

The Explore subagent

During plan mode (and other discovery-heavy phases), the **Explore subagent** isolates verbose discovery output and **returns a summary**, preserving the main conversation's context. Use it to prevent context-window exhaustion in multi-phase tasks.

Combining the two

A common, correct pattern: **plan first, then execute** — use plan mode to investigate and design (e.g., a library migration), then switch to direct execution to implement the chosen plan.

Exam trap: When the requirements *already* state large scale and architectural decisions (e.g., “re-structure the monolith into microservices across dozens of files”), the answer is **enter plan mode**

now — not “start direct and switch if it gets complex” (the complexity is already known), not “direct with comprehensive upfront instructions” (you don’t yet know the right structure). (See Worked Question 5.)

4.5 Apply iterative refinement techniques for progressive improvement (Task 3.5)

When prose instructions produce inconsistent results, several techniques get you to reliable output:

- **Concrete input/output examples.** The single most effective way to communicate an expected transformation. When natural-language descriptions are interpreted inconsistently, provide **2–3 concrete input→output pairs**.
- **Test-driven iteration.** Write a test suite first (expected behavior, edge cases, performance), then iterate by **sharing the test failures** with Claude to guide progressive fixes.
- **The interview pattern.** Have Claude **ask you questions** to surface considerations you may not have anticipated (cache-invalidation strategy, failure modes) *before* implementing — especially valuable in unfamiliar domains.
- **One message vs. sequential.** Provide **all issues in a single detailed message when the fixes interact** (so Claude can reconcile them together); fix **sequentially when the problems are independent**.

Why it matters: “Be more accurate” rarely helps. *Showing* Claude two correct examples, or handing it a failing test, communicates the standard far more precisely than adjectives. The exam favors concrete examples and tests over vague exhortations.

4.6 Integrate Claude Code into CI/CD pipelines (Task 3.6)

Running Claude Code in automation requires non-interactive operation and machine-parseable output.

The flags

- **-p / --print** — run in **non-interactive mode**: process the prompt, print the result to stdout, exit. This prevents the job from **hanging while waiting for interactive input** — the classic CI failure.
- **--output-format json** with **--json-schema** — produce **structured, machine-parseable findings** that a pipeline can post as inline PR comments.

Project context for CI

CLAUDE.md provides the **project context** that CI-invoked Claude Code needs: testing standards, fixture conventions, review criteria. Documenting these improves test-generation quality and reduces low-value output.

Avoiding duplicate and self-review problems

- **Re-running reviews after new commits:** include the **prior review findings** in context and instruct Claude to **report only new or still-unaddressed issues**, so you don’t post duplicate comments.
- **Test generation:** provide **existing test files** in context so Claude doesn’t suggest duplicates of scenarios already covered.
- **Session isolation matters:** the *same* Claude session that generated code is **less effective at reviewing its own changes** than an **independent review instance** (it carries the generation reasoning and is biased toward its own decisions). This is a recurring reliability theme — see Domain 4.6.

Exam trap: “The CI job hangs waiting for input” → add **-p** (the documented non-interactive flag). It is *not* a fictional `CLAUDE_HEADLESS` env var, not a `--batch` flag (which doesn’t exist for this), and not a stdin redirect workaround. (See Worked Question 10.)

Part 5 — Domain 4: Prompt Engineering & Structured Output (20%)

This domain is about getting **reliable, well-shaped output**: precise criteria that reduce false positives, few-shot examples that pin down format and judgment, JSON schemas that guarantee structure, validation loops that recover from errors, batch processing for scale, and multi-pass review for quality. The unifying idea: **be specific and show, don't just tell — and use the API mechanism that guarantees what you need rather than hoping a prompt achieves it.**

5.1 Design prompts with explicit criteria to improve precision and reduce false positives (Task 4.1)

Specific criteria beat vague instructions

Vague instructions (“check that comments are accurate”) produce vague behavior. **Explicit, categorical criteria** (“flag a comment **only when** the claimed behavior contradicts the actual code behavior”) produce precise behavior. The difference is the gap between an opinion and a rule.

Critically, **general hedges do not help precision**. Telling the model to “be conservative” or “only report high-confidence findings” does **not** reduce false positives the way **specific categorical criteria** do. (And as you will see in 5.2 and 5.5, the model’s *self-reported* confidence is poorly calibrated, so leaning on it backfires.)

Why false positives are so costly

High false-positive categories undermine trust in the accurate ones. If a code-review bot cries wolf in one category, developers start ignoring *all* its findings — including the correct ones. Protecting precision is protecting adoption.

Skills you must demonstrate

- **Write specific review criteria** that define which issues to **report** (bugs, security) vs. **skip** (minor style, local patterns) — rather than relying on confidence-based filtering.
- **Temporarily disable** a high-false-positive category to restore developer trust while you improve its prompt.
- **Define explicit severity criteria with concrete code examples** for each level to get consistent classification.

Exam trap: To fix poor precision, choose **explicit categorical criteria (with examples)** — not “tell it to be conservative,” not “raise the confidence threshold.” Specificity is the lever; confidence-based filtering is not.

5.2 Apply few-shot prompting to improve output consistency and quality (Task 4.2)

Few-shot (multishot) prompting — putting a few worked examples in the prompt — is the **most effective technique** for consistent, actionable output when detailed instructions alone fall short. Anthropic’s guidance: include **3–5 diverse, relevant, clear examples**, wrapped in `<example>` tags (nested in `<examples>` when there are several). More good examples generally means better performance.

What few-shot examples do for you: - **Demonstrate format** (location, issue, severity, suggested fix) so output is consistent. - **Show how to handle ambiguous cases** — e.g., which tool to pick for an ambiguous request, or how to judge a branch-level test-coverage gap — by **showing the reasoning** for why one choice was made over a plausible alternative. - **Enable generalization** — done well, examples teach *judgment* that transfers to novel patterns, rather than matching only the exact cases shown. - **Reduce hallucination in extraction** — examples covering varied document structures (inline citations vs. bibliographies, narrative vs. tables) cut down empty/null or fabricated fields.

Skills you must demonstrate

- Create **2–4 targeted examples for ambiguous scenarios** that show the *reasoning* for the chosen action.
- Include examples that **distinguish acceptable patterns from genuine issues** to reduce false positives while still generalizing.
- Use examples to **demonstrate handling of varied document formats** to fix empty/null extraction of required fields.

Why it matters: Examples *show* the standard. The exam repeatedly frames few-shot as the right answer for **format consistency**, **ambiguous-case judgment**, and **false-positive reduction** — but note the important exception from 1.4: when a rule must be *guaranteed* (financial gates), few-shot is **not** enough; you need programmatic enforcement.

5.3 Enforce structured output using tool use and JSON schemas (Task 4.3)

Tool use as the reliable path to JSON

The most reliable way to get **schema-compliant structured output** is **tool use** (`tool_use`) with a **JSON schema**: define an “extraction tool” whose `input_schema` is your desired structure, force the model to call it, and read the structured data from the `tool_use` block. This **eliminates JSON syntax errors** — the output is constrained to match the schema’s shape.

Current reality: Anthropic later added a dedicated **Structured Outputs** feature that compiles your JSON schema into a grammar to *guarantee* compliance (JSON-outputs mode and strict tool use). The tool-use approach the exam emphasizes remains valid and is still the canonical pattern; the newer feature is an even stronger guarantee where supported.

`tool_choice` recap (for output control)

- **auto** — model may return text *or* call a tool.
- **any** — model must call *some* tool (guarantees structured output when multiple extraction schemas exist and the document type is unknown).
- **forced** (`{"type": "tool", "name": "extract_metadata"}`) — model must call *that* tool (use to

ensure a specific extraction runs *before* enrichment steps).

Syntax errors vs. semantic errors

This is a subtle, testable distinction. **Strict schemas eliminate *syntax* errors but not *semantic* errors.** The output will be valid JSON of the right shape, but it can still be *wrong*: line items that don't sum to the stated total, a value placed in the wrong field, a fabricated number. Schemas guarantee structure, not correctness.

Schema design

- **Required vs. optional/nullable fields.** Make a field **optional/nullable** when the source may not contain it — this prevents the model from fabricating a value just to satisfy a required field.
- **Enums with an escape hatch.** Use enum fields plus an **"other"** value with a detail string for extensible categories, and an **"unclear"** value for genuinely ambiguous cases.
- **Normalization rules in the prompt.** Pair a strict output schema with **format-normalization instructions** in the prompt to handle inconsistent source formatting.

Remember: Tool use + JSON schema = guaranteed *structure*, no syntax errors. It does **not** guarantee the values are *semantically* correct — so you still validate (Domain 4.4). Nullable fields prevent hallucinated values; enums with "other"/"unclear" handle the messy real world.

5.4 Implement validation, retry, and feedback loops for extraction quality (Task 4.4)

Schemas catch syntax problems; **validation** catches semantic ones. The pattern:

- **Retry-with-error-feedback.** On a validation failure, send a follow-up request containing the **original document**, the **failed extraction**, and the **specific validation error(s)**, guiding the model to correct itself.
- **Know retry's limits.** Retries work for **format mismatches and structural output errors**. They are **useless when the required information is simply absent** from the source document — no amount of retrying conjures data that isn't there. Recognizing "the info exists only in an external document we didn't provide" saves wasted calls.

Designing self-correcting validation

- Extract **calculated_total** alongside **stated_total** so a mismatch is automatically flagged.
- Add a **conflict_detected** boolean for inconsistent source data.
- Add a **detected_pattern** field to findings so you can analyze *which* code constructs trigger findings — enabling systematic study of false-positive/dismissal patterns when developers reject findings.

Exam trap: Distinguish **semantic validation errors** (values don't sum, wrong field) from **schema syntax errors** (already eliminated by tool use). And remember retries fix *format/structure* problems, not *missing-information* problems.

5.5 Design efficient batch processing strategies (Task 4.5)

The Message Batches API

- **~50% cost savings** on both input and output tokens.

- **Up to a 24-hour processing window** (most batches finish in under an hour), with **no guaranteed latency SLA**.
- **custom_id** fields **correlate request/response pairs** — results come back **out of order**, so you match on custom_id.
- Large capacity (up to ~100k requests or 256 MB per batch); results retained ~29 days; **no webhooks** (you **poll** for completion); expired requests aren't billed but yield no result.
- **No multi-turn tool calling within a single request** — a batch request cannot execute tools mid-request and feed results back.

When to use it (and when not)

- **Appropriate** for **non-blocking, latency-tolerant** workloads: overnight reports, weekly audits, nightly test generation, bulk document extraction, offline evals.
- **Inappropriate** for **blocking** workflows where someone waits on the result — e.g., a **pre-merge check** that gates a developer. Use the **synchronous API** for those.

Skills you must demonstrate

- **Match the API to the latency requirement**: synchronous for blocking pre-merge checks, batch for overnight/weekly analysis.
- **Calculate submission frequency from the SLA** — e.g., submit every 4 hours to guarantee a 30-hour SLA given a 24-hour batch window.
- **Handle failures**: resubmit only the failed documents (found by custom_id), with fixes such as chunking documents that exceeded context limits.
- **Refine prompts on a sample first** before batch-processing large volumes, to maximize first-pass success and reduce costly resubmission.

Exam trap: When a proposal is to move *everything* to batch for the cost savings, the right answer is usually **batch only the non-blocking jobs (overnight reports) and keep synchronous calls for the blocking ones (pre-merge checks)**. Don't switch blocking workflows to batch "because it's usually fast" — there is no SLA. (See Worked Question 11.)

5.6 Design multi-instance and multi-pass review architectures (Task 4.6)

Self-review is weak

A model that generated something **retains its reasoning context** and is therefore **less likely to question its own decisions** in the same session. **Independent review instances** — fresh, without the generator's reasoning — catch subtle issues **better than** self-review instructions or even extended thinking.

Multi-pass review

For large reviews, **split into per-file local-analysis passes plus separate cross-file integration passes** (the same attention-dilution fix from Domain 1.6). This avoids superficial coverage and contradictory findings.

Skills you must demonstrate

- Use a **second, independent Claude instance** to review generated code without the generator's context.
- Split large multi-file reviews into **focused per-file passes** (local issues) plus **integration passes** (cross-file data flow).

- Run **verification passes** where the model reports confidence alongside each finding to enable **calibrated review routing** (route the low-confidence/ambiguous ones to humans).

Why it matters: “Have the model double-check itself” sounds appealing but is the *weaker* option. The exam favors an **independent reviewer** over self-review, and **focused multi-pass** review over one big pass.

Part 6 — Domain 5: Context Management & Reliability (15%)

This domain is about keeping systems **accurate and trustworthy over long, complex, multi-agent work**: preserving the facts that matter across long conversations, escalating wisely, propagating errors so the system can recover, surviving large-codebase exploration, routing the right cases to humans, and never losing track of *where information came from*.

6.1 Manage conversation context to preserve critical information across long interactions (Task 5.1)

The two failure modes

- **Progressive summarization risk.** As a long conversation is repeatedly summarized to save space, **precise facts get blurred** — numbers, percentages, dates, and customer-stated expectations turn into vague phrases. The \$47.50 refund the customer was promised becomes “a refund.”
- **“Lost in the middle.”** Models reliably attend to information at the **beginning and end** of a long input but may **drop findings from the middle**. Where you place information affects whether it is used.

Also, **tool results accumulate and consume tokens disproportionately to their relevance** — an order lookup might return 40+ fields when only 5 matter, and those 40 fields sit in context forever.

The techniques

- **Persistent “case facts” block.** Extract transactional facts (amounts, dates, order numbers, statuses) into a **persistent facts block included in every prompt**, *outside* the summarized history — so they never get blurred away. For multi-issue sessions, keep a separate structured layer of issue data (order IDs, amounts, statuses).
- **Trim verbose tool outputs** to only the relevant fields **before** they accumulate (keep just the return-relevant fields from an order lookup).
- **Position-aware ordering.** Put **key-findings summaries at the beginning** of aggregated inputs and organize details under **explicit section headers** to mitigate the lost-in-the-middle effect.
- **Structured outputs from subagents.** Require subagents to include **metadata** (dates, source locations, methodological context) and to **return structured data (key facts, citations, relevance scores) instead of verbose prose** when downstream agents have limited context budgets.
- **Pass complete history** in subsequent API requests to maintain conversational coherence (the API is stateless; you resend the messages).

Remember: Don’t let critical numbers ride inside summarized history. Pull them into a persistent, always-included facts block. Trim fat tool outputs early. Put the important stuff at the top and bottom.

6.2 Design effective escalation and ambiguity resolution patterns (Task 5.2)

When to escalate

Appropriate triggers are: - The **customer explicitly asks for a human**. - A **policy exception or gap** — the policy doesn't cover the request (not merely that the case is “complex”). - **Inability to make meaningful progress**.

When *not* to escalate

- Don't escalate just because a case *seems* hard or the customer *sounds* upset. **Sentiment-based escalation and self-reported confidence scores are unreliable proxies for actual complexity.** (The model is often *confidently wrong* on hard cases — so its confidence cannot gate escalation.)

The nuances the exam tests

- **Honor an explicit human request immediately** — do not first attempt your own investigation.
- But when the issue is **straightforward and within your capability, acknowledge any frustration and offer to resolve it, escalating only if the customer reiterates** the preference for a human.
- **Escalate on policy gaps** — e.g., the customer wants competitor price-matching but policy only addresses your own-site adjustments. The policy is *silent*, so escalate.
- **Multiple matches require clarification.** If a tool returns several possible customer matches, **ask for additional identifiers** — never pick one by heuristic.

How to implement

Add **explicit escalation criteria with few-shot examples** to the system prompt showing when to escalate vs. resolve.

Exam trap: The fix for poor escalation calibration is **explicit criteria + few-shot examples** — *not* self-reported confidence thresholds (poorly calibrated), *not* a separately trained classifier (over-engineered before prompt optimization is tried), and *not* sentiment analysis (sentiment $\neq$ complexity). (See Worked Question 3.)

6.3 Implement error propagation strategies across multi-agent systems (Task 5.3)

This is Domain 2.2's error principle applied across agents. When a subagent fails, the **coordinator** needs enough information to recover.

Good propagation

Return **structured error context**: the **failure type, what was attempted** (the query), any **partial results**, and **potential alternative approaches**. With this, the coordinator can decide to retry with a modified query, try an alternative, or proceed with partial results and annotate gaps.

The anti-patterns

- **Generic statuses** (“search unavailable”) **hide valuable context** from the coordinator and block informed decisions.
- **Silently suppressing errors** (returning empty results marked as success) prevents any recovery and risks shipping incomplete output.
- **Terminating the entire workflow on a single failure** throws away work when recovery could have

succeeded.

Skills you must demonstrate

- **Distinguish access failures from valid empty results** so the coordinator reacts correctly.
- Have subagents **recover locally** from transient failures and only **propagate what they cannot resolve**, with attempts and partial results attached.
- Structure synthesis output with **coverage annotations** — which findings are well-supported vs. which topic areas have gaps due to unavailable sources.

Exam trap: For a timed-out subagent, the best choice is **structured error context to the coordinator** — not a generic “unavailable” status (hides context), not “mark the empty result as success” (suppresses the failure), and not “terminate the whole workflow” (over-reacts). (See Worked Question 8.)

6.4 Manage context effectively in large codebase exploration (Task 5.4)

The degradation problem

In extended sessions, **context degrades**: the model starts giving inconsistent answers and **references “typical patterns”** instead of the **specific classes it discovered earlier**. That is a signal it has lost the concrete details.

The toolkit

- **Scratchpad files.** Have agents **write key findings to a scratchpad file** and reference it for later questions, persisting knowledge across context boundaries to counteract degradation.
- **Subagent delegation.** Spawn subagents to investigate specific questions (“find all test files,” “trace the refund-flow dependencies”) while the **main agent keeps the high-level coordination** and stays lean.
- **Summarize-then-spawn.** Summarize the key findings of one exploration phase **before** spawning sub-agents for the next, injecting the summary into their initial context.
- **Structured state persistence for crash recovery.** Each agent **exports its state to a known location**; on resume, the **coordinator loads a manifest** and injects it into agent prompts. This makes the system recoverable after a crash.
- **/compact.** Reduce context usage during extended exploration when context fills with verbose discovery output.

Remember: The cure for context degradation is **externalizing memory** — scratchpads, summaries injected into fresh contexts, subagents that absorb the verbose work, and manifests for crash recovery. When the model starts speaking in generalities about your code, it has lost the specifics; reach for these tools.

6.5 Design human review workflows and confidence calibration (Task 5.5)

Aggregate metrics lie

A headline “97% accurate” can **mask poor performance on a specific document type or field**. You must **validate accuracy by segment** — per document type and per field — *before* automating high-confidence

extractions. The whole can look healthy while a part is broken.

The techniques

- **Stratified random sampling.** Sample high-confidence extractions in a stratified way to **measure ongoing error rates and detect novel error patterns** — you keep watching even the cases you trust.
- **Field-level confidence, calibrated.** Have the model output **field-level confidence scores**, then **calibrate the review threshold using a labeled validation set**. Calibration against ground truth is what makes confidence usable for routing (recall that *raw* self-reported confidence is unreliable — calibration is the missing ingredient).
- **Route the risky ones to humans.** Send **low-confidence or ambiguous/contradictory** extractions to human review, **prioritizing limited reviewer capacity** where it matters most.

Why it matters: “We hit 97% overall, let’s automate” is a trap if one document type sits at 70%. Segment the accuracy, sample the trusted cases continuously, and calibrate confidence against labeled data before pulling humans out of the loop.

6.6 Preserve information provenance and handle uncertainty in multi-source synthesis (Task 5.6)

Provenance gets lost in summarization

When findings are compressed without preserving **claim-source mappings**, attribution disappears — you end up with a claim and no idea where it came from. The fix is to require **structured claim-source mappings** (source URLs, document names, relevant excerpts) that the synthesis agent must **preserve and merge**, not discard, when combining findings.

Handling conflict and time

- **Conflicting statistics from credible sources** should be **annotated with source attribution**, not silently resolved by arbitrarily picking one value. Let the coordinator decide how to reconcile.
- **Temporal data** must carry **publication/collection dates** so that differences over *time* are not misread as *contradictions* (two figures can both be correct for different years).

Skills you must demonstrate

- Require subagents to output **structured claim-source mappings** that survive synthesis.
- Structure reports to **distinguish well-established findings from contested ones**, preserving original source characterizations and methodological context.
- Have document analysis **complete with conflicting values included and explicitly annotated**, leaving reconciliation to the coordinator.
- **Render content types appropriately** — financial data as tables, news as prose, technical findings as structured lists — rather than forcing everything into one uniform format.

Remember: Never let a claim float free of its source. Carry URLs/excerpts/dates through every summarization step. Annotate conflicts and timestamps instead of resolving or hiding them. Distinguish “established” from “contested” in the final report.

Part 7 — The Six Scenarios Decoded

Every exam question lives inside one of these six scenarios. The exam shows you **four of the six**, chosen at random. For each, this part lays out the setup, the domains it draws on, the architectural decisions that matter, and the pitfalls the questions probe. Read these to learn how the abstract concepts combine into the concrete situations you will face.

Scenario 1 — Customer Support Resolution Agent

Setup. An Agent SDK agent handles high-ambiguity requests (returns, billing disputes, account issues) using MCP tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`. Target: **80%+ first-contact resolution**, with smart escalation.

Domains in play: Agentic Architecture & Orchestration, Tool Design & MCP, Context Management & Reliability.

Key decisions and patterns: - **Deterministic prerequisites** — block `process_refund` until `get_customer` returns a *verified* ID. A prompt is not enough where money is involved (Domain 1.4). - **Hooks for compliance** — a `PreToolUse` hook blocks refunds over a threshold (e.g., \$500) and redirects to escalation (Domain 1.5). - **PostToolUse normalization** — unify heterogeneous formats (timestamps, status codes) across these tools before the agent reasons (Domain 1.5). - **Tool descriptions** — `get_customer` vs. `lookup_order` must be clearly differentiated or the agent misroutes order queries to the customer tool (Domain 2.1). - **Structured errors** — categorize failures and tag retryability so the agent recovers well and explains business errors to the customer (Domain 2.2). - **Escalation calibration** — explicit criteria + few-shot; honor explicit human requests immediately; escalate on policy gaps; ask for more identifiers on multiple matches; ignore sentiment/self-confidence as triggers (Domain 5.2). - **Persistent case facts** — keep amounts/dates/order numbers in a facts block, not buried in summarized history; trim verbose order lookups (Domain 5.1). - **Structured human handoffs** — give the human ID, root cause, refund amount, recommended action, since they can't see the transcript (Domain 1.4).

Pitfalls the questions probe: choosing prompt instructions where a *gate/hook* is required; using sentiment or confidence to drive escalation; letting precise figures get summarized away; auto-selecting among multiple customer matches.

Scenario 2 — Code Generation with Claude Code

Setup. A team uses Claude Code for generation, refactoring, debugging, and documentation, integrated into their workflow with custom slash commands and `CLAUDE.md`, choosing plan mode vs. direct execution.

Domains in play: Claude Code Configuration & Workflows, Context Management & Reliability.

Key decisions and patterns: - **CLAUDE.md scope** — shared standards go to **project-level** (committed); personal preferences to **user-level**; more specific wins (Domain 3.1). - **Commands/skills scope** — team-wide commands in `.claude/commands/` (now skills in `.claude/skills/`); personal in `~/.claude/...`; use context: `fork`, `allowed-tools`, `argument-hint` (Domain 3.2). - **Path rules** — `.claude/rules/` with glob paths for conventions that span directories (Domain 3.3). - **Plan vs. direct** — plan mode for large/architectural multi-file work; direct execution for single-file fixes; combine (plan then execute); use the Explore subagent to keep context lean (Domain 3.4). - **Iterative refinement** — concrete input/output

examples, test-driven iteration, the interview pattern; one message for interacting fixes, sequential for independent ones (Domain 3.5).

Pitfalls the questions probe: putting shared config in user scope; reaching for plan mode on trivial changes (or *not* using it when complexity is already known); using per-directory CLAUDE.md for files scattered across the tree.

Scenario 3 — Multi-Agent Research System

Setup. A coordinator delegates to specialized subagents — one searches the web, one analyzes documents, one synthesizes, one generates reports — producing comprehensive, **cited** reports.

Domains in play: Agentic Architecture & Orchestration, Tool Design & MCP, Context Management & Reliability.

Key decisions and patterns: - **Hub-and-spoke** — coordinator routes all communication, decomposes, delegates, aggregates, and dynamically selects which subagents to run (Domain 1.2). - **Decomposition breadth** — partition scope to cover the whole topic and minimize duplication; iterative refinement loops to close gaps (Domain 1.2). - **Explicit context passing** — subagents don't inherit history; pass prior findings directly in prompts; separate content from metadata to preserve attribution (Domain 1.3). - **Parallel spawning** — multiple Task calls in one response; allowedTools includes Task (Domain 1.3). - **Scoped tools** — the synthesis agent shouldn't have web-search tools; give it a narrow `verify_fact` for simple checks and route complex verification through the coordinator (Domain 2.3, 5.3). - **Error propagation** — structured error context on a search timeout; proceed with partial results and annotate coverage gaps (Domain 5.3). - **Provenance** — claim-source mappings survive synthesis; annotate conflicts and dates; distinguish established vs. contested findings; render content types appropriately (Domain 5.6).

Pitfalls the questions probe: blaming downstream agents when the *coordinator's decomposition* was too narrow; over-provisioning the synthesis agent; suppressing or genericizing errors; losing citations in summarization.

Scenario 4 — Developer Productivity with Claude

Setup. An Agent SDK tool helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate, and automate chores, using built-in tools (Read, Write, Bash, Grep, Glob) and MCP servers.

Domains in play: Tool Design & MCP, Claude Code Configuration & Workflows, Agentic Architecture & Orchestration.

Key decisions and patterns: - **Right built-in tool** — Grep for content, Glob for filenames, Read+Write when Edit can't find a unique anchor; explore incrementally (Grep entry points → Read to follow imports) rather than reading everything (Domain 2.5). - **MCP integration** — `.mcp.json` (project) vs. `~/.claude.json` (personal); `${VAR}` for secrets; resources as catalogs to avoid exploratory calls; enhance MCP tool descriptions so they beat built-ins; reuse community servers (Domain 2.4). - **Context in big codebases** — scratchpads, subagent delegation for verbose exploration, summarize-then-spawn, manifests for crash recovery, `/compact` (Domain 5.4). - **Task decomposition** — dynamic decomposition for open-ended tasks like “add comprehensive tests to a legacy codebase” (Domain 1.6).

Pitfalls the questions probe: confusing Grep vs. Glob; reading all files upfront; committing secrets instead of using env expansion; letting context degrade instead of externalizing memory.

Scenario 5 — Claude Code for Continuous Integration

Setup. Claude Code is embedded in a CI/CD pipeline running automated code reviews, generating test cases, and giving PR feedback. Prompts must be **actionable** and **minimize false positives**.

Domains in play: Claude Code Configuration & Workflows, Prompt Engineering & Structured Output.

Key decisions and patterns: - **Non-interactive execution** — `-p / --print` to avoid hangs; `--output-format json + --json-schema` for machine-parseable findings posted as inline comments (Domain 3.6). - **CLAUDE.md for CI context** — testing standards, fixtures, review criteria (Domain 3.6). - **No duplicate comments** — include prior findings; report only new/unaddressed issues; provide existing tests so generation avoids duplicates (Domain 3.6). - **Independent review** — a separate instance reviews generated code better than self-review; split big reviews into per-file + integration passes (Domain 4.6, 1.6). - **Precision** — explicit categorical criteria (report bugs/security, skip minor style), severity criteria with examples; temporarily disable high-false-positive categories; few-shot examples distinguishing acceptable patterns from real issues (Domain 4.1, 4.2). - **Batch vs. sync** — batch the overnight technical-debt report; keep the blocking pre-merge check synchronous (Domain 4.5).

Pitfalls the questions probe: the hanging-job fix (must be `-p`); switching blocking checks to batch; using “be conservative”/confidence filtering instead of explicit criteria; one giant review pass causing contradictory feedback.

Scenario 6 — Structured Data Extraction

Setup. A system extracts information from unstructured documents, validates with JSON schemas, maintains high accuracy, handles edge cases, and integrates downstream.

Domains in play: Prompt Engineering & Structured Output, Context Management & Reliability.

Key decisions and patterns: - **Tool use + JSON schema** — the reliable structured-output path; `tool_choice`: any when the document type is unknown; forced tool to run a specific extraction first (Domain 4.3). - **Schema design** — nullable/optional fields to prevent fabrication; `enum + "other"/"unclear"`; normalization rules in the prompt (Domain 4.3). - **Syntax vs. semantic** — schemas kill syntax errors, not semantic ones (totals that don't sum); add `calculated_total` vs. `stated_total`, `conflict_detected` (Domain 4.3, 4.4). - **Validation/retry** — feed back specific validation errors with the document and failed extraction; know retries fail when info is simply absent (Domain 4.4). - **Few-shot for variety** — examples across document structures (inline citations vs. bibliographies, narrative vs. tables) to fix empty/null extraction (Domain 4.2). - **Batch at scale** — submit 100s of documents; handle failures by `custom_id`; chunk oversized docs; refine on a sample first (Domain 4.5). - **Human review** — field-level confidence calibrated on labeled data; stratified sampling of high-confidence extractions; segment accuracy by document type/field before automating (Domain 5.5).

Pitfalls the questions probe: assuming schemas guarantee *correct* values; retrying when info is absent; trusting a single aggregate accuracy number; required fields forcing fabrication.

Part 8 — Worked Sample Questions & Test-Taking Strategy

These twelve questions mirror the format and difficulty of the exam. **Decide your answer before reading the explanation.** Each explanation goes beyond “why the right answer is right” to “why each distractor is tempting but wrong” — because recognizing the trap is the skill the exam tests.

Question 1 — Skipped verification (Scenario 1)

In 12% of cases the agent skips `get_customer` and calls `lookup_order` using only the customer’s stated name, sometimes misidentifying accounts and issuing incorrect refunds. What most effectively fixes this reliability issue?

- A) A **programmatically prerequisite** that blocks `lookup_order` and `process_refund` until `get_customer` returns a verified customer ID.
- B) Enhance the system prompt to say verification is mandatory before order operations.
- C) Add few-shot examples showing the agent always calling `get_customer` first.
- D) A routing classifier that enables only the appropriate subset of tools per request.

Correct: A. A required tool *sequence* tied to critical business logic (verify identity before money moves) demands **deterministic** enforcement. A prerequisite gate guarantees it every time. **B and C are probabilistic** — prompts and examples reduce the failure rate but cannot eliminate it, which is unacceptable when refunds are wrong. **D** addresses *which tools are available*, not *what order they run in* — it solves a different problem.

Lesson: Guaranteed compliance → programmatic enforcement (gate/hook), not prompting. (*Domain 1.4*)

Question 2 — Tool misrouting (Scenario 1)

The agent calls `get_customer` for order questions (“check my order #12345”) instead of `lookup_order`. Both tools have minimal descriptions and accept similar identifiers. Best first step?

- A) Add 5–8 few-shot examples routing order queries to `lookup_order`.
- B) **Expand each tool’s description** with input formats, example queries, edge cases, and when to use it vs. the other.
- C) A routing layer that parses input and pre-selects the tool by keywords.
- D) Consolidate both into one `lookup_entity` tool.

Correct: B. Tool descriptions are the **primary** mechanism for selection; minimal descriptions are the root cause, and expanding them is a low-effort, high-leverage fix. **A** adds token overhead without fixing the root cause. **C** over-engineers and bypasses the model’s language understanding. **D** is a valid architecture choice but heavier than a *first step* when the real problem is thin descriptions.

Lesson: Fix selection problems at the description first. (*Domain 2.1*)

Question 3 — Escalation calibration (Scenario 1)

The agent resolves only 55% on first contact (target 80%). It escalates straightforward cases and tries to autonomously handle complex policy-exception cases. Best fix?

- A) **Explicit escalation criteria in the system prompt with few-shot examples** of when to escalate vs. resolve.
- B) Self-reported confidence score (1–10), auto-escalate below a threshold.
- C) A separate classifier trained on historical tickets.
- D) Sentiment analysis that escalates on negative sentiment.

Correct: A. The root cause is **unclear decision boundaries**, fixed directly by explicit criteria + examples — the proportionate first move. **B** fails because LLM self-confidence is poorly calibrated (the agent is already confidently wrong on hard cases). **C** is over-engineered before prompt optimization is even tried. **D** solves the wrong problem — **sentiment** ≠ **complexity**.

Lesson: Calibrate behavior with explicit criteria + few-shot; don't trust self-confidence or sentiment. (Domain 5.2)

Question 4 — Where to put a shared command (Scenario 2)

You want a `/review` slash command available to every developer when they clone or pull the repo. Where does the command file go?

- A) **`.claude/commands/` in the project repository.**
- B) `~/ .claude/commands/` in each developer's home directory.
- C) The root `CLAUDE.md`.
- D) A `.claude/config.json` with a commands array.

Correct: A. Project-scoped commands in `.claude/commands/` are version-controlled and automatically available to everyone who clones the repo. **B** is *personal*, not shared. **C** is for instructions/context, not command definitions. **D** describes a mechanism that **doesn't exist**.

Lesson: Shared = project scope (committed). Personal = user scope. (Domain 3.2)

Question 5 — Plan mode vs. direct (Scenario 2)

You must restructure a monolith into microservices — dozens of files, decisions about service boundaries and dependencies. Which approach?

- A) **Enter plan mode** to explore, understand dependencies, and design before changing anything.
- B) Direct execution incrementally, letting implementation reveal boundaries.
- C) Direct execution with comprehensive upfront instructions for each service.
- D) Start direct, switch to plan mode only if unexpected complexity appears.

Correct: A. Plan mode is *built for* large-scale, multi-approach, architectural, multi-file work — it enables safe exploration and design before committing, preventing costly rework. **B** risks discovering dependencies too late. **C** assumes you already know the right structure. **D** ignores that the complexity is **already stated**, not something that *might* emerge.

Lesson: Known architectural complexity → plan mode up front. (Domain 3.4)

Question 6 — Conventions for scattered test files (Scenario 2)

Different areas have different conventions; test files live alongside the code they test (`Button.test.tsx` next to `Button.tsx`) and you want all tests to follow the same conventions regardless of location. Most maintainable way to apply conventions automatically?

- A) `.claude/rules/` files with **YAML frontmatter glob patterns** (path-based conditional loading).
- B) Consolidate everything in root `CLAUDE.md` under headers, relying on inference.
- C) Skills in `.claude/skills/` for each code type.
- D) A separate `CLAUDE.md` in each subdirectory.

Correct: A. Glob path rules (e.g., `**/*.test.tsx`) apply conventions **by file type regardless of directory** — exactly right for files scattered across the tree, and they load only when relevant. **B** relies on inference (unreliable). **C** requires invocation, contradicting “automatic.” **D** can’t easily cover files spread across many directories (`CLAUDE.md` is directory-bound).

Lesson: Cross-directory, automatic, file-type conventions → glob path rules. (*Domain 3.3*)

Question 7 — Narrow decomposition (Scenario 3)

On “impact of AI on creative industries,” every subagent succeeds, yet the report covers only visual arts and misses music, writing, and film. The coordinator’s logs show subtasks: “AI in digital art,” “AI in graphic design,” “AI in photography.” Most likely root cause?

- A) The synthesis agent lacks gap-identification instructions.
- B) **The coordinator’s task decomposition is too narrow**, so subagent assignments miss whole domains.
- C) The web-search agent’s queries aren’t comprehensive enough.
- D) The document-analysis agent filters out non-visual sources.

Correct: B. The logs reveal it directly: the coordinator carved “creative industries” into only visual-arts subtasks. The subagents executed *correctly* — the problem is **what they were assigned**. **A, C, D** blame downstream agents that are working correctly within their scope.

Lesson: When every step succeeds but coverage is incomplete, suspect the coordinator’s decomposition. (*Domain 1.2*)

Question 8 — Error propagation on timeout (Scenario 3)

The web-search subagent times out. How should the failure flow back to the coordinator to enable intelligent recovery?

- A) **Return structured error context** — failure type, attempted query, partial results, alternative approaches.
- B) Retry internally with backoff, then return a generic “search unavailable.”
- C) Catch the timeout and return an empty result marked successful.
- D) Propagate the exception to a top-level handler that terminates the whole workflow.

Correct: A. Structured context lets the coordinator decide to retry differently, try an alternative, or proceed with partial results. **B**’s generic status hides context. **C** suppresses the failure (success-as-failure), preventing recovery and risking incomplete output. **D** over-reacts, discarding recoverable work.

Lesson: Propagate rich, structured error context — never generic, suppressed, or fatal. (*Domain 5.3*)

Question 9 — Reducing verification round-trips (Scenario 3)

The synthesis agent often needs to verify claims. Today it returns to the coordinator, which invokes web search, then re-invokes synthesis — adding 2–3 round trips and 40% latency. 85% of verifications are simple fact-checks; 15% need deeper investigation. Best approach?

- A) Give synthesis a **scoped verify_fact** tool for simple lookups, while complex verifications still route through the coordinator.
- B) Batch all verification needs and send them to the coordinator at the end.
- C) Give synthesis access to all web-search tools so it never round-trips.
- D) Have the web-search agent pre-cache extra context around each source.

Correct: A. This is **least privilege**: satisfy the 85% common case cheaply with one narrow tool, keep the existing coordination for the hard 15%. **B** creates blocking dependencies (later synthesis steps may need earlier verified facts). **C** over-provisions, violating separation of concerns. **D** relies on speculative caching that can't reliably predict needs.

Lesson: Add a *narrow* cross-role tool for the common case; route complex cases through the coordinator. (Domain 2.3)

Question 10 — CI job hangs (Scenario 5)

Your pipeline runs `claude "Analyze this PR for security issues"` but hangs, waiting for interactive input. Correct way to run Claude Code in automation?

- A) Add `-p: claude -p "Analyze this PR for security issues"`.
- B) Set `CLAUDE_HEADLESS=true`.
- C) Redirect stdin from `/dev/null`.
- D) Add a `--batch` flag.

Correct: A. `-p / --print` is the documented non-interactive mode: process the prompt, print to stdout, exit. **B** and **D** reference **non-existent** features. **C** is a Unix workaround that doesn't properly address Claude Code's command model.

Lesson: Non-interactive Claude Code in CI = `-p / --print`. (Domain 3.6)

Question 11 — Batch vs. synchronous (Scenario 5)

Two workflows use real-time calls: (1) a blocking pre-merge check developers wait on, and (2) an overnight technical-debt report. A manager proposes moving both to the Batch API for 50% savings. How to evaluate?

- A) Use batch for the technical-debt reports only; keep real-time for pre-merge checks.
- B) Switch both to batch with status polling.
- C) Keep real-time for both to avoid ordering issues.
- D) Switch both to batch with a timeout fallback to real-time.

Correct: A. Batch saves 50% but has up to a 24-hour window and **no latency SLA** — unsuitable for a blocking check, ideal for an overnight job. **B** is wrong because "often fast" is not acceptable for blocking work. **C** misconceives the API — results correlate via `custom_id`, so ordering isn't a blocker. **D** adds needless complexity when matching each API to its use case is simpler.

Lesson: Batch the latency-tolerant jobs; keep blocking workflows synchronous. (Domain 4.5)

Question 12 — Inconsistent large review (Scenario 5)

A PR touches 14 files. A single-pass review gives uneven depth, misses obvious bugs, and even contradicts itself (flagging a pattern in one file, approving identical code in another). How to restructure?

- A) **Split into focused passes** — analyze each file individually, then a separate cross-file integration pass.
- B) Require developers to split large PRs into 3–4 file submissions.
- C) Switch to a larger-context-window model to fit all 14 files.
- D) Run three independent passes and only flag issues appearing in 2 of 3.

Correct: A. The root cause is **attention dilution** across many files at once; per-file passes ensure consistent depth, and a dedicated integration pass catches cross-file issues. **B** shifts burden to developers without improving the system. **C** misunderstands the problem — bigger context doesn't fix attention *quality*. **D** would *suppress* real bugs caught only intermittently by demanding consensus.

Lesson: For inconsistent multi-file reviews, split into per-file + integration passes. (Domains 1.6, 4.6)

Test-Taking Strategy

The mindset

Most questions follow a template: “Here is a production system misbehaving. What is the best fix?” Your job is to find the option that **addresses the root cause** with the **least unnecessary complexity** and the **right level of guarantee**.

A reliable decision procedure

1. **Identify the root cause.** What is *actually* broken — tool ordering, tool selection, decomposition, escalation boundaries, attention dilution, error reporting, context loss? Many distractors fix a *different* problem.
2. **Match the guarantee level.** Does the rule need to hold *every time* (money, safety, identity) or just *usually*? Guaranteed → programmatic (gate/hook/forced tool). Usually → prompt/few-shot.
3. **Prefer the proportionate fix.** The exam rewards the simplest effective change. “Add a description,” “add explicit criteria,” “split the passes” usually beat “build a classifier,” “add a routing layer,” “switch models.”
4. **Beware over-engineering distractors.** Trained classifiers, ML pipelines, new routing layers, and bigger models are frequently the *wrong* answer because a simpler prompt/config/tool change solves it.
5. **Distrust self-confidence and sentiment.** Any option that gates behavior on the model's *self-reported confidence* or on *customer sentiment* is almost always wrong (confidence is uncalibrated; sentiment ≠ complexity). Calibration against labeled data is the exception that makes confidence usable.
6. **Watch for “blame the wrong component.”** If every subagent/step succeeded but output is incomplete, the fault is upstream (the coordinator/decomposition), not the workers.
7. **Read the requirements literally.** If complexity is *already stated*, choose plan mode now; if a step is *required*, enforce it programmatically; don't defer to “switch later if needed.”

Answer every question

There is no guessing penalty, and blanks count as wrong. Eliminate the clearly-wrong options and commit.

Part 9 — Hands-On Preparation Exercises

The exam is explicitly designed to reward **practical judgment**, not memorization. The single most effective preparation is to *build* the systems the scenarios describe. This part takes the four exercises from the official guide and expands each into a complete, self-contained lab: what to build, the exact steps, **what success looks like**, the **mistakes to watch for**, and a short set of **self-check questions** that mirror how the exam probes each concept.

You do not need a large project to do these. A handful of scripts, a couple of fake “backend” functions, and a sample folder of documents are enough. The goal is to *feel* each tradeoff with your own hands so the exam’s distractors become obvious.

How to use this part: Do the exercises in order — each builds intuition the next one assumes. After each, answer the self-check questions out loud. If you can explain *why* the right approach beats the alternatives, you understand the domain.

Exercise 1 — Build a Multi-Tool Agent with Escalation Logic

Reinforces: Domain 1 (Agentic Architecture), Domain 2 (Tool Design & MCP), Domain 5 (Reliability).

Objective: Build a small agent — a customer-support helper is the classic example — that runs a real agentic loop, calls several tools, handles errors intelligently, enforces a business rule programmatically, and escalates when appropriate. This is essentially Scenario 1 in miniature, and it touches the highest-weighted domain on the exam.

Step 1 — Define 3-4 tools with carefully differentiated descriptions

Create tools such as `get_customer`, `lookup_order`, `process_refund`, and `escalate_to_human`. Deliberately include **two tools that are easy to confuse** (e.g., one that looks up customers and one that looks up orders, both accepting an ID-like string). Then write descriptions that make the boundary unmistakable: state the **purpose**, the **exact input format**, an **example query**, and an explicit “**use this instead of X when...**” clause.

Why it matters: Sample Question 2 turns on exactly this. When two tools have thin descriptions (“Retrieves customer information” / “Retrieves order details”) the model misroutes. The fix is richer descriptions, not a routing layer. Feel that difference: write thin descriptions first, watch the agent misroute, then expand them and watch it recover.

Step 2 — Implement the agentic loop on `stop_reason`

Write the loop: send the conversation to Claude, read `stop_reason`. If it is `tool_use`, execute the requested tool(s), append the results to the conversation as `tool_result` blocks, and loop again. If it is `end_turn`, stop and present the final message.

- Do drive the loop purely on `stop_reason`.

- **Don't** parse the assistant's prose for phrases like "I'm done," don't stop just because there's text content, and don't use a hard iteration cap as your *primary* stopping mechanism (a high cap as a *safety net* is fine).

Exam trap: Every one of those "don'ts" is a named anti-pattern in Domain 1.1. The loop terminates on `end_turn` — full stop.

Step 3 — Add structured error responses

Make your tools return **structured** errors, not bare strings. Include `errorCategory` (`transient` / `validation` / `business` / `permission`), an `isRetryable` boolean, and a human-readable message. Then test each path: - A **transient** error (simulate a timeout) → the agent should retry. - A **business** error (e.g., "refund exceeds policy") with `isRetryable: false` → the agent should *not* retry; it should explain to the user or escalate. - A **validation** error → the agent should fix the input and try again.

Why it matters: A generic "Operation failed" gives the model nothing to reason with, so it either gives up or retries pointlessly. Structured metadata is what lets the agent recover *intelligently* (Domain 2.2). Distinguish an **access failure** (retry-worthy) from a **valid empty result** (a successful query that simply found nothing) — conflating them is a classic bug.

Step 4 — Add a programmatic hook that enforces a business rule

Add a `PreToolUse`-style interception on `process_refund` that **blocks** the call when the amount exceeds a threshold (say \$500) and redirects to the escalation workflow. Confirm it fires *every single time*, regardless of how the prompt is phrased.

The single most-tested idea on this exam: when a rule must hold **deterministically** (money, identity, safety), enforce it in **code** (a hook or a prerequisite gate), not in the prompt. Prompt instructions have a non-zero failure rate. Add the same rule as a prompt instruction too, then try to talk the agent past it — you'll eventually succeed, which is exactly why the hook must exist.

Also implement a **prerequisite gate**: block `lookup_order` and `process_refund` until `get_customer` has returned a verified customer ID. This is the literal answer to Sample Question 1.

Step 5 — Test with multi-concern messages

Send a message bundling several issues ("My order #123 arrived broken **and** I was double-charged on my last invoice **and** I want to update my address"). Verify the agent **decomposes** the request into distinct items, investigates each (in parallel where possible, using shared verified-customer context), and **synthesizes one unified resolution** rather than handling only the first concern.

What success looks like

- The loop never hangs and never stops early; it ends precisely on `end_turn`.
- Transient errors are retried; business errors are explained, never retried.
- The refund hook blocks over-threshold refunds **100%** of the time.
- `get_customer` always runs before any order/refund operation.
- A three-part message yields a single answer addressing all three parts.

Common mistakes to avoid

- Driving loop termination off natural-language cues or text presence.
- Returning uniform/opaque errors that erase the transient-vs-business distinction.

- Enforcing the money/identity rule only in the prompt “because the model usually complies.”
- Treating an empty result as an error (or an error as an empty result).

Self-check

1. Your agent occasionally processes a \$900 refund despite a prompt forbidding it. What is the *only* reliable fix, and why isn’t a stronger prompt enough?
 2. A tool returns { "results": [] }. How should the agent tell “the search failed” apart from “the search worked and found nothing”?
 3. Why is a fixed max_turns cap the wrong *primary* stopping mechanism, and what is the right one?
-

Exercise 2 — Configure Claude Code for a Team Development Workflow

Reinforces: Domain 3 (Claude Code Configuration), Domain 2 (MCP Integration).

Objective: Stand up a realistic team configuration for Claude Code: a layered CLAUDE.md, path-scoped rules, a forked skill with restricted tools, and both shared and personal MCP servers. This is Scenario 2 made concrete and the heart of Domain 3.

Step 1 — Create a project-level CLAUDE.md

At the repo root (or .claude/CLAUDE.md), write **universal, always-true** standards: coding conventions, testing expectations, review criteria. Commit it. Confirm that a teammate who clones the repo inherits these instructions automatically.

Why it matters: Project-level config is **version-controlled and shared**; user-level config (~/.claude/CLAUDE.md) is **personal and never shared**. The exam loves the diagnostic case: “a new teammate isn’t getting the standards” → the rules were placed at user level, not project level (Domain 3.1).

Step 2 — Create path-scoped rules in .claude/rules/

Add rule files with YAML frontmatter paths globs so conventions load **only** when editing matching files:

- paths: ["src/api/**/*"] → API error-handling conventions.
- paths: ["**/*.test.*"] → testing conventions, which apply to test files **wherever they live**.

Edit a matching file and a non-matching file; confirm the rules activate only for matches.

Why it matters: This is the exact answer to Sample Question 6. When files of a kind (tests) are **scattered across directories**, glob-pattern rules beat both a monolithic CLAUDE.md (relies on the model *inferring* which section applies) and per-directory CLAUDE.md files (directory-bound, can’t follow scattered files). Path rules also save tokens by loading only when relevant (Domain 3.3).

Step 3 — Create a forked skill with restricted tools

In .claude/skills/<name>/SKILL.md, write a skill for something **verbose** — a codebase analysis or a brainstorming pass. Set frontmatter: - context: fork so it runs in an **isolated sub-agent** and its noisy output never pollutes your main conversation. - allowed-tools: limited to what it actually needs (e.g., read-only tools, or only file-write) to prevent destructive actions. - argument-hint: so invoking it without arguments prompts the developer for the parameter.

Run it and confirm the main conversation stays clean.

Why it matters: context: fork is the configuration-level cousin of the subagent isolation idea from Domain 1 — keep verbose work out of the main context. Note also the skills-vs-CLAUDE.md distinction (Domain 3.2): **skills** are on-demand, task-specific workflows; **CLAUDE.md** is always-loaded universal standards. *(In current Claude Code, slash commands and skills have converged on the SKILL.md format; the exam's Feb-2025 framing still treats .claude/commands/ as the slash-command location — see §4.2 for the reconciliation.)*

Step 4 — Configure shared and personal MCP servers

- In project-scoped `.mcp.json`, configure a shared server (e.g., a GitHub or Jira integration) using **environment-variable expansion** (`${GITHUB_TOKEN}`) so the token is never committed.
- In user-scoped `~/.claude.json`, add a personal/experimental server.

Start a session and confirm **both** servers' tools are available simultaneously, discovered at connection time.

Why it matters: Project vs. user scope mirrors the CLAUDE.md split — shared/committed vs. personal/uncommitted. `${VAR}` expansion is the standard secret-management pattern. And remember: enrich your MCP tool descriptions, or the agent may default to a built-in tool (like Grep) over a more capable MCP tool (Domain 2.4).

Step 5 — Compare plan mode and direct execution

Run three tasks and consciously pick a mode for each: - A **single-file bug fix** with a clear stack trace → **direct execution**. - A **library migration touching dozens of files** → **plan mode**. - A **new feature with multiple valid designs** → **plan mode**.

Observe where plan mode prevents costly rework and where it would just add ceremony.

Why it matters: Sample Question 5 (monolith → microservices) is decided here. When complexity, multiple valid approaches, or architectural decisions are **already stated in the requirements**, choose plan mode **now** — don't "start direct and switch later" (Domain 3.4).

What success looks like

- A fresh clone inherits project standards with zero setup.
- Test conventions apply to `Button.test.tsx` and `api/user.test.ts` alike, by glob.
- The forked skill's verbose output never appears in your main thread.
- Both MCP servers' tools are live at once; no secret is committed.
- You can justify plan-vs-direct for each of the three tasks in one sentence.

Common mistakes to avoid

- Putting team standards in `~/.claude/CLAUDE.md` (they won't be shared).
- Using per-directory CLAUDE.md for conventions that follow scattered files.
- Forgetting context: fork, letting a noisy skill bury the main conversation.
- Hard-coding tokens in `.mcp.json` instead of `${VAR}` expansion.
- Defaulting to plan mode for trivial fixes (over-ceremony) or to direct execution for clearly architectural work (costly rework).

Self-check

1. A teammate reports Claude ignores the team's testing rules. Name the two most likely configuration causes and how you'd confirm each.
2. Tests live next to their components throughout the tree. Why do glob path-rules beat both a root `CLAUDE.md` and per-directory `CLAUDE.md` files here?
3. A skill that explores the whole codebase keeps flooding your session. Which one frontmatter setting fixes it, and what does it do?

Exercise 3 — Build a Structured Data Extraction Pipeline

Reinforces: Domain 4 (Prompt Engineering & Structured Output), Domain 5 (Reliability).

Objective: Build a pipeline that extracts structured data from messy documents using `tool_use` + JSON schema, validates and retries intelligently, uses few-shot examples for format variety, runs at scale via the Batch API, and routes uncertain cases to humans. This is Scenario 6, and it exercises two full domains at once.

Step 1 — Define an extraction tool with a careful schema

Define a tool whose **input schema** is your extraction target. Include: - **Required** fields (always present) and **optional/nullable** fields (may be absent in the source). - An **enum** field with an **"other" + detail string** escape hatch, plus an **"unclear"** value for genuinely ambiguous cases.

Feed it documents where some fields are **absent** and confirm the model returns **null** instead of **fabricating** a value.

Why it matters: Two of the most-tested extraction ideas live here. (1) `tool_use` with a JSON schema gives **guaranteed schema-valid output** — it eliminates *syntax* errors. (2) Making genuinely-optional fields **nullable** is what stops the model from inventing data to satisfy a "required" field (Domain 4.3). The "other"/"unclear" enums keep rigid schemas from forcing wrong choices.

Step 2 — Build a validation-and-retry loop

After extraction, run **semantic** validation (e.g., with Pydantic): do line items sum to the stated total? Are values in the right fields? When validation **fails**, send a follow-up request that includes the **original document**, the **failed extraction**, and the **specific error**, asking for correction.

Crucially, **track which failures retries can fix**: - **Fixable by retry**: format mismatches, mis-placed values, structural output errors. - **Not fixable by retry**: the information **isn't in the document at all** — retrying just burns tokens.

Why it matters: Domain 4.4 draws a hard line between **semantic** errors (which tool-use does *not* prevent — schemas guarantee shape, not correctness) and **syntax** errors (which tool-use *does* eliminate). And it stresses knowing when retry is futile. Self-correction patterns — extracting `calculated_total` alongside `stated_total`, or a `conflict_detected` boolean — let the system *flag* discrepancies instead of hiding them.

Step 3 — Add few-shot examples for format variety

Add 2–4 examples showing extraction from **structurally different** documents — inline citations vs. a bibliography, a narrative paragraph vs. a table, formal vs. informal phrasing. Confirm previously-missed

fields now extract correctly.

Why it matters: Few-shot examples are the exam’s go-to remedy when **detailed instructions alone** still produce inconsistent output, and they’re specifically called out for **reducing hallucination and empty/null extractions** on varied document structures (Domain 4.2). Show the *reasoning* in tricky examples, not just the answer, so the model generalizes.

Step 4 — Design a batch processing strategy

Submit a batch of ~100 documents through the **Message Batches API**. Give each a unique **custom_id**. When results arrive (results come back **out of order** — correlate by **custom_id**), identify failures and **resubmit only those**, with fixes (e.g., **chunk** documents that blew past the context limit). Then do the SLA arithmetic: if a single batch can take up to 24 hours, what submission cadence guarantees, say, a 30-hour turnaround?

Why it matters: Domain 4.5 and Sample Question 11 hinge on this. Batch = ~50% **cheaper, up to a 24-hour window, no latency SLA, no multi-turn tool calling inside a request** → great for overnight/weekly jobs, **wrong for blocking pre-merge checks**. Refine your prompt on a small sample *before* spending a 100-doc batch, to maximize first-pass success.

Step 5 — Add human-review routing with calibrated confidence

Have the model emit a **field-level confidence** per extraction. Route **low-confidence** or **ambiguous/contradictory-source** extractions to human review. Then **measure**: analyze accuracy by document type and by field, and **calibrate** your thresholds against a **labeled validation set**. Use **stratified random sampling** of the *high-confidence* bucket to catch novel error patterns hiding behind a good aggregate number.

Why it matters: Domain 5.5 warns that a single headline accuracy (e.g., “97%”) can **mask** terrible performance on one document type or field. Raw model confidence is only trustworthy **after calibration against labeled data** — otherwise it’s the same uncalibrated self-confidence the exam tells you not to trust for escalation.

What success looks like

- Absent fields come back `null`; the model doesn’t invent values.
- Retries fix format/structural errors and are *skipped* when data is truly missing.
- Few-shot examples visibly improve extraction across document layouts.
- Failed batch items are resubmitted by **custom_id**, oversized ones chunked.
- Review routing is driven by **calibrated** confidence, validated per document type and field.

Common mistakes to avoid

- Marking genuinely-optional fields **required**, inviting fabrication.
- Assuming `tool_use` guarantees *correctness* — it guarantees *shape*; semantic validation is still your job.
- Retrying when the information simply isn’t in the source.
- Trusting **raw** confidence scores without calibrating against labeled data.
- Reporting only aggregate accuracy and missing a weak segment.

Self-check

1. Your schema marks `discount_code` required, and the model invents codes for documents that have none. What's the fix, and why does it work?
2. An extraction's line items don't sum to the total. Does `tool_use` with a strict schema catch this? Why or why not — and what *would* catch it?
3. Overall accuracy is 97%, but downstream users complain about invoices specifically. What measurement practice would have surfaced this before automation?

Exercise 4 — Design and Debug a Multi-Agent Research Pipeline

Reinforces: Domain 1 (Orchestration), Domain 2 (Tool Design), Domain 5 (Context, Provenance, Error Propagation).

Objective: Build the Scenario 3 system — a coordinator delegating to specialized subagents — and deliberately exercise its hardest parts: explicit context passing, parallel spawning, structured provenance, error propagation with partial results, and conflict handling in synthesis. This is the capstone; it pulls together more task statements than any other exercise.

Step 1 — Build a coordinator that delegates with explicit context

Create a coordinator and at least two subagents (e.g., **web search** and **document analysis**). Ensure the coordinator's `allowedTools` includes **"Task"** — without it, it cannot spawn subagents at all. Pass each subagent the findings it needs **directly in its prompt**; do **not** assume it inherits the coordinator's history.

Why it matters: Subagents run with **isolated context** — only their final result returns to the coordinator, and they share no memory between invocations (Domain 1.3). Forgetting "Task" in `allowedTools` is the literal reason a coordinator "can't delegate." Pass the synthesis agent the *actual* search results and analysis outputs, not a vague pointer to them.

Step 2 — Spawn subagents in parallel

Have the coordinator emit **multiple Task calls in a single response** to run subagents concurrently, and compare latency against issuing them across separate turns.

Why it matters: Parallelization is an explicit skill in Domain 1.3 and a core "Building Effective Agents" pattern. Emitting the calls **together** is what makes them parallel; one-per-turn is just sequential with extra steps. (*Note the depth limit: subagents cannot themselves spawn subagents — only the top-level coordinator spawns.*)

Step 3 — Design structured output that separates content from metadata

Make each subagent return findings as **structured records**: a `claim`, an evidence excerpt, a source (URL / document name), and a `publication_date`. Verify the **synthesis agent preserves attribution** when it merges everything.

Why it matters: Provenance is **lost during summarization** when you compress findings into prose without keeping **claim→source mappings** (Domain 5.6). Keeping content and metadata in separate structured fields is what survives synthesis. It also reduces context bloat — downstream agents

get key facts, not verbose reasoning chains (Domain 5.1).

Step 4 — Implement error propagation with partial results

Simulate a **web-search subagent timeout**. The subagent should first attempt **local recovery** for transient failures; if it still fails, it should propagate **structured error context** — failure type, attempted query, **partial results**, and possible alternatives — back to the coordinator. Confirm the coordinator can **proceed with partial results** and **annotate the final report with coverage gaps** rather than crashing or silently returning empty.

Why it matters: This is Sample Question 8 exactly. The right shape is **structured error context**, not a generic “search unavailable” (hides information), not an empty-set-marked-success (silent suppression), not killing the whole workflow on one failure (Domain 5.3 / 2.2). The two named anti-patterns: **silent suppression** and **whole-workflow termination**.

Step 5 — Handle conflicting sources in synthesis

Feed the system **two credible sources with different statistics**. Verify synthesis **keeps both values with attribution** instead of silently picking one, and that the report **separates well-established findings from contested ones**. Add **publication dates** so a *temporal* difference (2019 figure vs. 2024 figure) isn’t mistaken for a *contradiction*.

Why it matters: Domain 5.6 is explicit: **annotate conflicts with source attribution** rather than arbitrarily choosing; preserve methodological context; require dates so temporal gaps aren’t misread as disagreement. Render content types appropriately too — financial data as tables, news as prose — instead of flattening everything into one format.

Bonus — Diagnose the “narrow decomposition” failure

Recreate Sample Question 7: run the system on a broad topic (e.g., “impact of AI on creative industries”) but have the coordinator decompose it into only one sub-area (e.g., three flavors of *visual* art). Observe that **every subagent succeeds** yet the **report is incomplete**. Fix it at the **coordinator/decomposition** level — not by blaming the (correctly-working) downstream agents.

Exam trap: When all workers succeed but coverage is incomplete, the fault is **upstream decomposition**. Add iterative refinement: have the coordinator evaluate synthesis for **gaps** and re-delegate targeted queries until coverage is sufficient.

What success looks like

- The coordinator can spawn subagents (“Task” present) and passes context explicitly.
- Parallel Task calls measurably cut latency versus sequential turns.
- Every claim in the final report traces to a source; conflicts show both values.
- A subagent timeout yields partial results + a coverage-gap annotation, never a crash or a silent empty.
- A too-narrow run is diagnosed as a **coordinator** problem and fixed there.

Common mistakes to avoid

- Omitting “Task” from `allowedTools`, then wondering why delegation fails.
- Assuming subagents inherit the coordinator’s context.

- Summarizing away source attribution before synthesis.
- Returning generic errors, empty-as-success, or terminating on one failure.
- Blaming downstream agents when the **decomposition** was too narrow.

Self-check

1. A coordinator “can’t delegate to its subagents.” What’s the first configuration thing to check, and what concept explains why subagents also need findings passed in their prompt?
 2. The web-search subagent times out. Describe the error payload that lets the coordinator recover intelligently, and name the two anti-patterns it avoids.
 3. Two reputable sources report different market-size numbers. What should synthesis do, and what extra field prevents misreading a *temporal* gap as a contradiction?
 4. Every subagent succeeded but the report misses whole sub-topics. Where is the bug, and what loop would prevent it next time?
-

A suggested study sequence

If you want a path through these labs that mirrors the exam’s weighting:

1. **Start with Exercise 1.** It anchors the highest-weighted domain (agentic loops, hooks, errors) and the single most-tested idea: *deterministic enforcement in code for must-hold rules*.
2. **Do Exercise 4 next.** It deepens orchestration and adds provenance and error propagation — together with Exercise 1 this covers most of Domains 1, 2, and 5.
3. **Then Exercise 3.** It owns Domain 4 (structured output, retries, batch) and reinforces calibration from Domain 5.
4. **Finish with Exercise 2.** It locks in Domain 3 (Claude Code configuration) and the MCP-scoping half of Domain 2.

After all four, re-read **Part 8** and confirm you can name the root cause and the proportionate fix for every sample question without looking. Then complete the official practice exam referenced in the guide.

Part 10 — Quick Reference

This part is your last-mile review. Skim 10.1 to confirm vocabulary, drill 10.2 until the tables are automatic, internalize 10.3 because those patterns *are* the correct answers, and use 10.4 to read primary sources for anything still fuzzy.

10.1 Master Glossary

Terms are grouped by theme. Each entry is written to be exam-ready: what it is, and the one thing the exam expects you to know about it.

Agents, loops, and orchestration

- **Agentic loop** — The repeating cycle: send the conversation to Claude → inspect `stop_reason` → if `tool_use`, run the tool(s) and append results → loop → stop on `end_turn`. The loop's control flow is driven by `stop_reason`, nothing else.
- **stop_reason** — The field on a model response saying *why* it stopped. Key values: **tool_use** (Claude wants to call a tool — continue the loop), **end_turn** (Claude is finished — stop), **max_tokens** (hit the output limit), **refusal** (declined on safety grounds).
- **tool_result** — The block you append to the conversation carrying a tool's output back to the model, so it can reason about the next step. Tool results accumulate in context.
- **Model-driven decision-making** — Claude decides which tool to call next based on context, as opposed to a **pre-configured decision tree**. Agentic systems favor the former.
- **Workflow vs. agent** — A **workflow** follows predefined code paths; an **agent** lets the model direct its own steps and tool use. Start with the simplest thing that works; add agentic autonomy only when needed.
- **Coordinator (orchestrator)** — The hub agent that decomposes a task, decides which subagents to invoke, routes all inter-subagent communication, aggregates results, and handles errors. Central to the hub-and-spoke pattern.
- **Subagent** — A specialized agent the coordinator spawns for a sub-task. Runs with **isolated context** (no automatic inheritance of the coordinator's history), and **only its final result returns**.
- **Hub-and-spoke architecture** — All communication flows through the coordinator (the hub), never directly between subagents (the spokes). Gives observability, consistent error handling, and controlled information flow.
- **Task tool (a.k.a. Agent tool)** — The mechanism a coordinator uses to spawn a subagent. The coordinator's **allowedTools must include "Task"** or it cannot delegate. (*Current SDK versions also expose this as the Agent tool; the exam's Feb-2025 framing calls it Task.*)
- **AgentDefinition** — The configuration for a subagent type: its description, system prompt, allowed tools, and (in current SDKs) model and reasoning effort.
- **Parallel spawning** — Emitting **multiple Task calls in a single coordinator response** so subagents run concurrently. One-per-turn is merely sequential.
- **Subagent depth limit** — A subagent **cannot spawn its own subagents**; only the top-level coordinator spawns. (*Current-platform detail; reinforces the single-hub model.*)
- **Iterative refinement loop** — The coordinator evaluates synthesis output for **gaps**, re-delegates targeted queries to search/analysis subagents, and re-invokes synthesis until coverage is sufficient.
- **Narrow-decomposition failure** — When subagents all succeed but the output misses whole sub-

topics because the **coordinator decomposed the task too narrowly**. The fault is upstream, not in the workers.

Enforcement, hooks, and handoffs

- **Programmatic enforcement** — Guaranteeing behavior in **code** (hooks, prerequisite gates) rather than prompt text. Required whenever a rule must hold **deterministically** (money, identity, safety).
- **Prompt-based guidance** — Steering behavior with instructions/examples. Probabilistic — it has a **non-zero failure rate**, so it's insufficient for must-hold rules.
- **Hook** — A deterministic callback that fires at a defined point in the agent lifecycle, runs in the application process, and costs no model tokens. Exam-relevant hooks: **PreToolUse** (intercept/block an outgoing tool call), **PostToolUse** (transform a tool result before the model sees it), plus **UserPromptSubmit**, **Stop**, **SubagentStart/SubagentStop**, **PreCompact**.
- **Prerequisite gate** — A block that prevents a downstream tool call until a prerequisite step has completed (e.g., block `process_refund` until `get_customer` returned a verified ID).
- **Data normalization (via PostToolUse)** — Using a hook to convert heterogeneous tool outputs (Unix timestamps, ISO-8601, numeric status codes) into one consistent format before the model reasons over them.
- **Structured handoff** — A compiled escalation summary (customer ID, root cause, amount, recommended action) handed to a human who **lacks the conversation transcript**.

Tools and MCP

- **Model Context Protocol (MCP)** — An open standard (announced Nov 2024) for connecting AI applications to external systems over **JSON-RPC 2.0**. Architecture: **hosts** contain **clients**, each client connects to a **server**.
- **MCP server primitives** — **Tools** (model-controlled — *actions* the model chooses to invoke), **Resources** (application-controlled — *context/data*, e.g., content catalogs), **Prompts** (user-controlled — *templates* the user triggers). Knowing *who controls which* is exam gold.
- **MCP resource** — A way to expose a **content catalog** (issue summaries, doc hierarchies, DB schemas) so the agent has visibility **without** burning exploratory tool calls.
- **isError flag** — The MCP mechanism for signaling a tool failure back to the agent within a tool result.
- **Structured error response** — An error payload carrying `errorCategory` (transient/validation/business/permission), an `isRetryable/retryable` boolean, and a human-readable message — so the agent recovers intelligently.
- **Transient vs. validation vs. business vs. permission error** — Transient = retry (timeout, unavailability); validation = fix input and retry; business = policy violation, **don't** retry, explain/escalate; permission = not allowed.
- **Access failure vs. valid empty result** — A failed query (needs a retry decision) vs. a successful query that found nothing (no retry needed). Never conflate them.
- **Tool description** — The **primary** signal the model uses to choose a tool. Thin descriptions cause misrouting among similar tools; rich descriptions (purpose, input format, examples, edge cases, “use vs. similar tool” boundaries) fix it.
- **Tool splitting vs. consolidation** — Split a vague generic tool into purpose-specific tools with clear contracts; consolidation is valid but heavier and rarely the “first step.”
- **tool_choice** — Controls whether/which tool the model calls: **auto** (may answer in text or call a tool), **any** (must call *some* tool), **forced** `{"type": "tool", "name": "..."} (must call that tool), and none (may not call any tool). (The exam's appendix lists three — auto, any, forced; current APIs add none.)`
- **Scoped tool access (least privilege)** — Give each agent only the tools its role needs. Too many tools (e.g., 18 vs. 4–5) degrades selection; off-role tools get misused.

- **.mcp.json** — **Project-scoped**, shared/version-controlled MCP server config. Supports `${VAR}` **environment-variable expansion** for secrets.
- **~/ .claude.json** — **User-scoped**, personal/experimental MCP server config (not shared).
- **Built-in tools** — **Grep** (search file *contents* for patterns), **Glob** (match file *paths*/names, e.g., `**/*.test.tsx`), **Read/Write** (full-file ops), **Edit** (targeted change via *unique* text match; fall back to Read+Write when the anchor isn't unique), **Bash**. Build understanding incrementally (Grep to find entry points → Read to follow imports), not by reading everything upfront.

Claude Code configuration

- **CLAUDE.md hierarchy** — **User-level** (`~/ .claude/CLAUDE.md`, personal, *not* shared), **project-level** (`.claude/CLAUDE.md` or root `CLAUDE.md`, shared via version control), **directory-level** (subdirectory files). More-specific scopes layer onto broader ones.
- **@import** — Syntax to pull external files into a `CLAUDE.md`, keeping it modular (import only the standards relevant to each package).
- **.claude/rules/** — Directory of topic-specific rule files (e.g., `testing.md`) as an alternative to one monolithic `CLAUDE.md`. With YAML frontmatter **paths** globs, rules load **only** when editing matching files.
- **Path-scoped rule** — A rule activated by glob (`paths: [ "**/*.test.*" ]`) regardless of directory — the right tool for conventions that follow **scattered** files.
- **Slash command** — A reusable command. **Project-scoped** in `.claude/commands/` (shared); **user-scoped** in `~/ .claude/commands/` (personal). (*In current Claude Code, commands and skills have converged on the `SKILL.md` format.*)
- **Skill / SKILL.md** — An on-demand, task-specific workflow defined in `.claude/skills/<name>/SKILL.md`. Frontmatter: **context: fork** (run isolated in a sub-agent, keep verbose output out of the main thread), **allowed-tools** (restrict tools during the skill), **argument-hint** (prompt for missing args).
- **Skill vs. CLAUDE.md** — Skill = **on-demand**, task-specific; `CLAUDE.md` = **always-loaded**, universal standards.
- **/memory** — Command to see which memory/config files are loaded; used to diagnose inconsistent behavior.
- **/compact** — Command to compress conversation context during long sessions.
- **Plan mode** — For complex, large-scale, multi-file, architecturally-significant, or multiple-valid-approach tasks: explore and design **before** changing code, preventing costly rework.
- **Direct execution** — For simple, well-scoped changes (single-file fix, one validation check).
- **Explore subagent** — Isolates verbose discovery output and returns a summary, preserving the main conversation's context budget.
- **-p / --print** — CLI flag for **non-interactive** mode (process prompt → print → exit). Essential in CI so jobs don't hang waiting for input.
- **--output-format json / --json-schema** — CLI flags to enforce **machine-parseable structured output** in CI (e.g., to post findings as inline PR comments).
- **Session resumption (--resume <name>)** — Continue a specific prior named conversation.
- **fork_session** — Branch independently from a shared analysis baseline to explore divergent approaches in parallel.
- **Resume vs. fresh-summary** — Resume when prior context is mostly valid; start fresh with an injected **structured summary** when prior tool results are **stale** (e.g., files changed since).

Prompting and structured output

- **Explicit criteria** — Specific, categorical rules (“flag a comment only when claimed behavior contradicts actual code”) beat vague guidance (“check comments are accurate”) and beat “be conservative”/“only high-confidence.”

- **False-positive management** — High-false-positive categories erode trust in *accurate* categories; temporarily disable a noisy category while you fix its prompt.
- **Few-shot prompting** — 2–4 targeted, diverse examples; the most effective fix when detailed instructions still yield inconsistent output. Show *reasoning* on ambiguous cases so the model **generalizes** rather than pattern-matching only the given cases. Reduces hallucination/empty extractions on varied formats.
- **tool_use for structured output** — Defining a tool whose input schema is your target gives **guaranteed schema-valid** output and **eliminates JSON syntax errors**.
- **Syntax vs. semantic errors** — Tool-use/strict schemas remove **syntax** errors (malformed JSON, wrong shape) but **not semantic** errors (line items don't sum, value in wrong field). Semantic validation is still required.
- **Nullable/optional fields** — Mark fields the source may lack as nullable so the model returns **null** instead of **fabricating** values.
- **"other" + detail / "unclear" enums** — Escape hatches that keep rigid enums from forcing a wrong category on extensible or ambiguous data.
- **Validation-and-retry loop** — On failure, resend the **document + failed extraction + specific error** for self-correction. Effective for format/structural errors; **futile when the info is absent** from the source.
- **Self-correction fields** — Extract `calculated_total` beside `stated_total`, or add `conflict_detected`, to **flag** discrepancies rather than hide them.
- **Strict mode / Structured Outputs** — Schema-as-grammar enforcement of output shape. (*The exam references strict JSON schemas via tool use; current platforms also offer a dedicated Structured Outputs feature — same goal: guaranteed shape.*)

Batch, review, and reliability

- **Message Batches API** — ~50% cheaper, up to 24-hour processing window, **no latency SLA, no multi-turn tool calling within a request**. Right for overnight/weekly latency-tolerant jobs; **wrong for blocking** (pre-merge) workflows.
- **custom_id** — Per-request identifier to correlate batch responses (which return **out of order**) and to **resubmit only failed** items.
- **Batch SLA arithmetic** — Size your submission cadence around the 24-hour ceiling (e.g., 4-hour windows to back a ~30-hour SLA).
- **Self-review limitation** — A model that just generated something retains its reasoning and is **less likely to question itself**. Prefer an **independent** review instance.
- **Multi-pass review** — Split large reviews into **per-file** local passes plus a **cross-file integration** pass to avoid **attention dilution** and contradictory findings. A bigger context window does **not** fix attention quality.
- **Confidence calibration** — Raw model confidence is unreliable until **calibrated against a labeled validation set**; only then use it to route review.
- **Stratified random sampling** — Sample within segments (e.g., high-confidence extractions) to measure true error rates and catch novel patterns an aggregate metric hides.

Context management, escalation, and provenance

- **Progressive-summarization risk** — Summaries that blur away exact numbers, percentages, dates, and customer-stated expectations. Avoid by keeping facts verbatim.
- **"Lost in the middle"** — Models attend reliably to the **start and end** of long inputs and may **drop middle** content. Put key findings **first** and use explicit section headers.
- **Case-facts block** — A persistent block of transactional facts (amounts, dates, order numbers, statuses) included in **every** prompt, kept **outside** summarized history.
- **Tool-output trimming** — Strip verbose tool results to only the relevant fields (e.g., 5 of 40+ order

fields) before they bloat context.

- **Scratchpad file** — A file where an agent records key findings, re-read later to counteract context degradation across boundaries.
- **Context degradation** — In long sessions, models drift to “typical patterns” instead of the specific classes discovered earlier; counter with scratchpads, summaries, subagent isolation, /compact.
- **Crash-recovery manifest** — Each agent exports state to a known location; the coordinator loads a **manifest** on resume and re-injects it.
- **Escalation triggers** — Escalate on an **explicit human request** (immediately, no investigation first), a **policy gap/exception** (policy silent or ambiguous), or **inability to make meaningful progress** — *not* on complexity alone.
- **Sentiment/self-confidence are bad escalation proxies** — Customer sentiment $\neq$ case complexity; self-reported confidence is uncalibrated. Use **explicit criteria + few-shot** instead.
- **Multiple-match disambiguation** — When a lookup returns several matches, **ask for another identifier** rather than guessing heuristically.
- **Error propagation** — Subagents handle transient failures **locally**; propagate only unrecoverable errors **with** failure type, what was attempted, and **partial results**. Anti-patterns: **silent suppression** (empty-as-success) and **whole-workflow termination** on one failure.
- **Coverage annotations** — Synthesis marks which findings are well-supported vs. which topic areas have **gaps** from unavailable sources.
- **Claim-source mapping (provenance)** — Structured links (claim $\rightarrow$ source URL/name $\rightarrow$ excerpt) that downstream agents must **preserve** through synthesis, so attribution survives.
- **Conflict annotation** — When credible sources disagree, **keep both values with attribution**; separate well-established from contested findings.
- **Temporal data** — Require **publication/collection dates** so a time gap (2019 vs. 2024 figure) isn’t misread as a contradiction.
- **Content-type rendering** — Render financial data as tables, news as prose, technical findings as lists — don’t flatten everything to one format.

10.2 Decision Tables and Cheat Sheets

Drill these until you can reproduce the right-hand column on sight. They compress most of the exam’s decision points.

stop_reason — what to do

Value	Meaning	Loop action
tool_use	Claude wants to call a tool	Execute tool(s), append tool_result, continue
end_turn	Claude has finished	Stop , present final response
max_tokens	Output limit reached	Handle truncation (e.g., continue or raise limit)
refusal	Declined on safety grounds	Stop; do not retry to force compliance

tool_choice — when to use which

Setting	Behavior	Use when
auto	May answer in text or call a tool	Default; tool use is optional
any	Must call some tool (model picks)	You need structured output / a tool <i>guaranteed</i> , type unknown
forced { <code>"type": "tool", "name": "..."</code> }	Must call that tool	A specific step must run first (e.g., <code>extract_metadata</code> before enrichment)
none	May not call any tool	Force a plain-text turn (<i>current APIs; beyond the exam's listed three</i>)

Hooks — interception points

Hook	Fires	Typical use
PreToolUse	Before a tool call executes	Block policy-violating calls (refund > \$500), gate prerequisites
PostToolUse	After a tool returns, before the model sees it	Normalize heterogeneous data (timestamps, status codes)
UserPromptSubmit	On user input	Inject context, validate input
Stop / SubagentStop	When the (sub)agent finishes	Cleanup, logging, state export
PreCompact	Before automatic compaction	Preserve critical facts before compression

Hooks run in the app process, are deterministic, and cost no model tokens.

MCP server primitives — who controls them

Primitive	Controlled by	Purpose	Example
Tools	Model	Actions the model chooses to take	<code>process_refund</code> , <code>web_search</code>
Resources	Application	Context/data exposed to the model	Issue catalog, DB schema, doc hierarchy
Prompts	User	Reusable templates the user triggers	<code>"/summarize-ticket"</code> template

CLAUDE.md scope — shared or personal?

Scope	Location	Shared via VCS?	Use for
User	<code>~/.claude/CLAUDE.md</code>	No (personal)	Your own preferences
Project	<code>.claude/CLAUDE.md</code> or root <code>CLAUDE.md</code>	Yes	Team standards, conventions

Scope	Location	Shared via VCS?	Use for
Directory	subdirectory <code>CLAUDE.md</code>	Yes	Area-specific rules (directory-bound)

Diagnostic reflex: “teammate isn’t getting the rules” → they’re at **user** scope; move to **project** scope.

Where does this configuration live?

You want to...	Put it in...
Share a slash command with the team	<code>.claude/commands/</code> (<i>project</i>)
Keep a personal command	<code>~/.claude/commands/</code>
Apply conventions to scattered files by type	<code>.claude/rules/</code> with <code>paths: globs</code>
Set always-on team standards	<code>project CLAUDE.md</code>
Define an on-demand task workflow	<code>.claude/skills/<name>/SKILL.md</code>
Share an MCP server (with secrets)	<code>.mcp.json</code> + <code>\${VAR}</code> expansion
Add a personal/experimental MCP server	<code>~/.claude.json</code>

Plan mode vs. direct execution

Choose plan mode when...	Choose direct execution when...
Large-scale / multi-file changes	Single-file, well-scoped change
Multiple valid approaches	One obvious approach
Architectural decisions (service boundaries)	A clear bug fix with a stack trace
Complexity is already stated	Adding one validation check

Don’t “start direct and switch later” when the requirements already say it’s complex.

Batch API vs. synchronous API

Dimension	Message Batches API	Synchronous API
Cost	~50% cheaper	Standard
Latency	Up to 24h , no SLA	Immediate
Tool calling	No multi-turn within a request	Yes
Result order	Out of order (use <code>custom_id</code>)	In order
Right for	Overnight reports, weekly audits, bulk extraction	Blocking pre-merge checks, interactive use

Built-in tool selection

Goal	Tool
Find files by name/extension (<code>**/*.test.tsx</code>)	Glob
Search file <i>contents</i> (callers, error strings, imports)	Grep
Read a whole file	Read
Targeted edit via a unique anchor string	Edit
Edit when the anchor isn't unique	Read + Write (fallback)
Run a shell command	Bash

Prompt vs. programmatic enforcement

Need	Approach
Rule must hold every time (money, identity, safety)	Programmatic — hook / prerequisite gate / forced tool
Behavior should usually follow a pattern	Prompt — explicit criteria + few-shot
Guarantee output shape	<code>tool_use</code> + strict JSON schema
Guarantee output correctness	Schema plus semantic validation (+ retry)

Escalate vs. resolve

Situation	Action
Customer explicitly asks for a human	Escalate immediately (don't investigate first)
Policy is silent/ambiguous on the request	Escalate (policy gap)
Agent can't make meaningful progress	Escalate
Issue is straightforward & within capability	Resolve (acknowledge frustration, offer to help; escalate only if they reiterate)
Decision based on sentiment or self-confidence	× Unreliable proxy — use explicit criteria instead
Lookup returns multiple matches	Ask for another identifier — don't guess

10.3 The “Recurring Right Answers” Pattern Library

Across the sample questions and task statements, a small set of principles is correct **again and again**. If a question maps to one of these, the answer almost always follows the pattern. Memorize the patterns; recognize them under exam pressure.

1. For must-hold rules, enforce in code — not in the prompt. Money, identity verification, safety, required ordering → use a **hook**, **prerequisite gate**, or **forced tool**. Prompts have a non-zero failure rate. (*Sample Q1.*) A stronger-worded prompt is a tempting wrong answer.

2. Fix tool selection by improving tool descriptions first. When similar tools get misrouted, the low-effort, high-leverage fix is **richer descriptions** (purpose, inputs, examples, boundaries) — before few-shot examples, routing layers, or tool consolidation. (*Sample Q2.*)

3. **Fix decision boundaries with explicit criteria + few-shot.** Escalation miscalibration, false positives, inconsistent classification → add **explicit categorical criteria** and **few-shot examples** demonstrating the boundary. This is the proportionate first move before any ML/classifier. (*Sample Q3.*)
4. **Distrust self-reported confidence and customer sentiment.** Any option gating behavior on the model's **confidence score** or on **sentiment** is almost always wrong — confidence is uncalibrated and sentiment $\neq$ complexity. The exception: confidence **calibrated against labeled data**. (*Sample Q3; Domain 5.5.*)
5. **When all workers succeed but output is incomplete, blame the coordinator.** Incomplete coverage with healthy subagents = **too-narrow decomposition** upstream. Don't blame the (correctly-working) downstream agents. (*Sample Q7.*)
6. **Propagate errors as structured context with partial results.** On a subagent failure, return **failure type + attempted action + partial results + alternatives**. Never a generic status (hides info), never empty-as-success (silent suppression), never kill the whole workflow on one failure. (*Sample Q8; Domains 5.3, 2.2.*)
7. **Apply least privilege; add narrow scoped tools for the common case.** Don't over-provision an agent with every tool. Give a scoped tool for the frequent simple case (e.g., `verify_fact` for 85% of checks) and route the rare complex case through the coordinator. (*Sample Q9; Domain 2.3.*)
8. **Project scope = shared; user scope = personal.** Team-wide things (commands, standards, MCP servers) go in the **project** (`.claude/...`, `.mcp.json`, version-controlled). Personal things go at **user** scope (`~/.claude/...`, `~/.claude.json`). (*Sample Q4; Domains 3.1–3.2, 2.4.*)
9. **Scattered-by-type conventions → path-scoped rules with globs.** For files spread across directories (tests, configs), `.claude/rules/` with paths: globs beats a monolithic `CLAUDE.md` (inference) and per-directory `CLAUDE.md` (directory-bound). (*Sample Q6; Domain 3.3.*)
10. **Pick plan mode when complexity is already stated.** Architectural / multi-file / multiple-approach work → **plan mode now**. Don't "start direct, switch later." Simple, clear-scope work → direct execution. (*Sample Q5; Domain 3.4.*)
11. **In CI, run non-interactively with `-p / --print`.** Hanging CI job waiting for input → add `-p`. `CLAUDE_HEADLESS`, `--batch`, and stdin redirection are wrong/non-existent. (*Sample Q10; Domain 3.6.*)
12. **Match the API to the workload's latency need.** Batch (cheap, $\leq 24h$, no SLA, no in-request tools) for overnight/bulk; synchronous for blocking pre-merge checks. Don't batch a workflow developers wait on. (*Sample Q11; Domain 4.5.*)
13. **Split big reviews into per-file + integration passes.** Inconsistent/contradictory multi-file review = **attention dilution**. Per-file passes + a cross-file pass fix it; a bigger context window does not. (*Sample Q12; Domains 1.6, 4.6.*)
14. **`tool_use` guarantees shape, not correctness.** Strict JSON schemas eliminate **syntax** errors; **semantic** errors (sums, field placement) still need validation + retry. (*Domains 4.3–4.4.*)
15. **Make optional fields nullable to stop fabrication.** If the source may lack a field, mark it nullable so the model returns `null` instead of inventing a value. Add "other"/"unclear" enums for extensible/ambiguous cases. (*Domain 4.3.*)
16. **Retry only when retrying can help.** Format/structural errors → retry with the error fed back. Information **absent** from the source → retrying is futile; route to human or mark missing. (*Domain 4.4.*)
17. **Use an independent instance to review generated work.** A model that just generated code is biased toward its own reasoning; a **separate** review instance (no prior context) catches more. (*Domain 4.6.*)
18. **Preserve provenance through structured claim-source mappings.** Keep claim → source → excerpt

→ date as structured fields so attribution survives summarization; annotate conflicts with both values; use dates to avoid mistaking temporal gaps for contradictions. (*Domain 5.6.*)

19. Protect critical facts from summarization and position effects. Keep exact numbers/dates in a persistent **case-facts** block; put key findings **first** (counter “lost in the middle”); **trim** verbose tool outputs to relevant fields. (*Domain 5.1.*)

20. Subagents have isolated context; pass what they need, include "Task". Subagents don’t inherit history — pass findings **in the prompt**. A coordinator needs **"Task" in allowedTools** to delegate at all. (*Domain 1.3.*)

The meta-rule: Find the **root cause**, choose the **proportionate** fix, and match the **guarantee level**. Distractors love to (a) fix a *different* problem, (b) **over-engineer** (trained classifier, routing layer, bigger model) when a description/criteria/config change suffices, or (c) rely on **probabilistic** compliance where a **deterministic** guarantee is required.

10.4 Further Reading

These are the primary sources behind the exam’s technologies. The exam’s source guide is dated **Feb 2025 (Version 0.1)**; the certification publicly launched **March 12, 2026**, and the platform has evolved since the draft, so where this study guide flagged a difference (the Task/Agent tool naming, the commands→skills convergence, the fourth tool_choice value none, and the dedicated Structured Outputs feature), treat the **live documentation** as authoritative for current behavior while answering exam questions in the framing above.

Official certification (start here) - Registration and the authoritative, current exam details (question count, time, cost, scoring) live on Anthropic’s **Skilljar** site — search “Claude Certified Architect Foundations” from Anthropic’s training/Academy pages, or go to the access-request page linked from Anthropic’s certification announcements. - **Anthropic Academy** offers free preparation courses (covering Claude fundamentals, Claude Code, agentic architecture, MCP, and more) that are open to everyone — use these as your primary, vendor-authored study path alongside this guide.

Agent SDK (agent loop, subagents, hooks, sessions) - Agent SDK overview and guides — <https://docs.claude.com/en/api/agent-sdk/overview> (and the agent-sdk section generally) - Subagents, hooks, sessions, and permissions are covered within the Agent SDK docs.

Claude Code (CLAUDE.md, rules, skills, plan mode, CLI) - Claude Code documentation — <https://docs.claude.com/en/docs/claude-code> and <https://code.claude.com/docs> - Memory/CLAUDE.md, settings, skills, and the CLI reference (-p/--print, --output-format) live here.

Model Context Protocol (MCP) - Official MCP site and specification — <https://modelcontextprotocol.io> - Server primitives (tools, resources, prompts) and the architecture overview are in the spec. - Connecting MCP to Claude Code / the API — <https://docs.claude.com/en/docs/mcp>

Claude API (tool use, structured output, batches) - Tool use overview — <https://docs.claude.com/en/docs/build-with-claude/tool-use> - Structured outputs — <https://docs.claude.com/en/docs/build-with-claude/structured-outputs> - Message Batches API — <https://docs.claude.com/en/docs/build-with-claude/batch-processing>

Prompt engineering - Prompt engineering overview — <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview> - Multishot (few-shot) prompting — <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/multishot-prompting>

Anthropic engineering essays (the *why* behind the patterns) - *Building Effective Agents* — <https://www.anthropic.com/engineering/building-effective-agents> (workflows vs. agents; the five patterns: prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer) - *Writing Tools for Agents* — <https://www.anthropic.com/engineering/writing-tools-for-agents> (high-leverage tools, naming, namespacing, returning human-readable results, search > list)

If a URL has moved, search the docs hub at <https://docs.claude.com> for the topic name — the documentation is reorganized periodically, but these topics persist.

Final word

You now have, in one place: the exam’s structure and scoring (Part 0); the four foundational technologies (Part 1); a task-by-task walkthrough of all five domains (Parts 2–6); every scenario decoded (Part 7); every sample question worked with a reusable test-taking method (Part 8); four hands-on labs that build the exact systems the exam tests (Part 9); and a glossary, decision tables, pattern library, and source list for last-mile review (Part 10).

The exam rewards one habit above all: **diagnose the root cause, then choose the simplest fix that gives the right level of guarantee.** Build the four exercises, drill the §10.2 tables, internalize the §10.3 patterns, take the official practice exam, and you’ll be ready.

Good luck.

This study guide is an independent learning aid based on the Claude Certified Architect — Foundations exam guide and Anthropic’s public technical documentation. Product details current as of mid-2026; verify specifics against the live documentation linked in §10.4.